The

<#FREEMARKER>

User Guide

build business websites
everything you need

Overall structure

Templates are in fact programs you write in a language called **FTL** (for FreeMarker Template Language). This is a quite simple programming language designed for writing templates and nothing else.

A template (= FTL program) is a mix of the following sections:

- **Text**: Text that will be printed to the output as is.
- **Interpolation**: These sections will be replaced with a calculated value in the output. Interpolations are delimited by `${` and `}` (or with `#{` and `}`, but that shouldn't be used anymore; see more here).
- **FTL tags**: FTL tags are a bit similar to HTML tags, but they are instructions to FreeMarker and will not be printed to the output.
- **Comments**: Comments are similar to HTML comments, but they are delimited by `<#--` and `-->`. Comments will be ignored by FreeMarker, and will not be written to the output.

Let's see a concrete template. I have marked the template's components with colors: text, interpolation, FTL tag, comment. With the *[BR]*-s I intend to visualize the line breaks.

```
<html>[BR]
<head>[BR]
  <title>Welcome!</title>[BR]
</head>[BR]
<body>[BR]
  <#-- Greet the user with his/her name -->[BR]
  <h1>Welcome ${user}!</h1>[BR]
  <p>We have these animals:[BR]
  <ul>[BR]
  <#list animals as animal>[BR]
    <li>${animal.name} for ${animal.price} Euros[BR]
  </#list>[BR]
  </ul>[BR]
</body>[BR]
</html>
```

FTL distinguishes upper case and lower case letters. So `list` is good directive name, while `List` is not. Similarly `${name}` is not the same as `${Name}` or `${NAME}`

It is important to realize that interpolations can be used in text (and in string literal expressions; see later) only.

An FTL tag can't be inside another FTL tag nor inside an interpolation. For example this is *WRONG*: `<#if <#include 'foo'>='bar'>...</#if>`

Comments can be placed inside FTL tags and interpolations. For example:

```
<h1>Welcome ${user <#-- The name of user -->}!</h1>[BR]
<p>We have these animals:[BR]
<ul>[BR]
<#list <#-- some comment... --> animals as <#-- again... --> animal>[BR]
...
```

Note:

For those of you who have tried the above examples: You may notice that some of spaces, tabs and line breaks are missing from the template output, even though we said that text is printed as is. Don't bother with it now. This is because the feature called "white-space stripping" is turned on, and that automatically removes some superfluous spaces, tabs and line breaks. This will be explained later.

How to Reuse Template Fragments

5.1. Reuse common template fragments

When you find yourself copy-pasting common parts between templates a lot, you should probably use macros.

Continuing our last example, create a new folder called *lib* inside the *templates* directory, and there create a file called *utils.ftl*, with this content:

```
<#macro page>
  <html>
  <head>
    <title>${title}</title>
  </head>
  <body>
    <h1>${title}</h1>

    <#-- This processes the enclosed content:  -->
    <#nested>
  </body>
  </html>
</#macro>

<#macro otherExample p1 p2>
  <p>The parameters were: ${p1}, ${p2}</p>
</#macro>
```

Now you can simplify *helloworld.ftl* like this:

```
<#import "lib/utils.ftl" as u>

<@u.page>
  <p>${exampleObject.name} by ${exampleObject.developer}</p>

  <ul>
    <#list systems as system>
      <li>${system_index + 1}. ${system.name} from
${system.developer}</li>
    </#list>
  </ul>

  <#-- Just another example of using a macro: -->
  <@u.otherExample p1=11 p2=22 />
```

```
</@u.page>
```

Another way of reusing template fragments is moving the common fragment into its own ftl file. Then just insert it with `<#include "lib/myfragment.ftl">`. This is less flexible than macros, but simpler in concept: it mimics copy-pasting.

5.2. Variables

You can define and assign content to variables inside the FTL files for easy reuse.

```
<#assign var_link = "http://www.vogella.com/people/larsvogel.html">

<a href="${var_link}">About Lars Vogel</a>
```

5.3. Handling null/undefined values

FreeMarker requires you to provide an explicit default for variables, so avoid values that are `null` or undefined:

```
<!-- Acts like if the color was N/A if there's no color: -->
<p>Color: ${color!'N/A'}</p>

<!-- Avoid the whole color row if there's no color: -->
<#if color??>
  <p>Color: ${color}</p>
</#if>
```

5.4. Escape

When generating HTML, it's important to escape `<`, `&`, etc. in values that were not meant to store HTML and can contain these problematic characters. You can apply such escaping like `${message?html}`. You can also ask FreeMarker to add `?html` to all `${}` -s in a section like this:

The types

Page Contents

- Scalars
- Containers
- Subroutines
  - Methods and functions
  - User-defined directives
  - Function/method versus user-defined directive
- Miscellaneous
  - Nodes
  - Markup output

The suppored types are:

- Scalars:
  - String
  - Number
  - Boolean
  - Date-like (date, time, or date-time)
- Containers:
  - Hash
  - Sequence
  - Collection
- Subroutines:
  - Methods and functions
  - User-defined directives
- Miscellaneous/seldom used:
  - Node
  - Markup output

Scalars

These are the basic, simple kind of values. They can be:

- String: It is simple text, e.g., the name of a product.

  If you want to give a string value directly in the template, rather than use a variable that comes from the data model, you write the text between quotation marks, e.g., `"green mouse"` or `'green mouse'`. (More details regarding the syntax can be found later.)

- Number: For example the price of a product. *Whole numbers and non-whole numbers are not distinguished; there is only a single number type. So for example 3/2 will be always 1.5, and never 1. Just like if you are using a calculator.*

  If you want to give a numerical value directly in the template, then you write for example: `150` or `-90.05` or `0.001`. (More details regarding the syntax can be found later.)

- Boolean: A boolean value represents a logical true or false (yes or no). For example, if a the visitor has been logged in or not. Typically you use booleans as the condition of

the `if` directive, like `<#if loggedIn >...</#if>` or `<#if price == 0>...</#if>`; in the last case the result of the `price == 0` part is a boolean value.

In the templates you can directly specify a boolean with the reserved words `true` and `false`.

- Date: A date-like value stores date/time related data. It has three variations:
  - Date: Like April 4, 2003. Day precision, no time of day part.
  - Time: Like 10:19:18 PM. Millisecond precision, no date part.
  - Date-time (sometimes called "time stamp") as April 4, 2003 10:19:18 PM. Both date and time, with millisecond precision.

  Unfortunately, because of the limitations of the Java platform, FreeMarker sometimes can't decide which parts of the date are in use (i.e., if it is date-time, a date or a time). The solution for this problem is an advanced topic that will be discussed later.

  It is possible to define date-like values directly in templates, but this is an advanced topic that will be explained later.

Bear in mind that FreeMarker distinguishes strings from numbers, booleans and date-like values. For example, while the string `"150"` looks like the number `150`, a string is still just arbitrary sequence of characters, and you can't do arithmetic with it, can't compare it with another number, etc.

Containers

These are the values whose purpose is to contain other variables; they are just containers. The contained variables are often referred as *sub variables*. The container types are:

- Hash: Associates a unique lookup name with each of its sub variables. The name is an unrestricted string. A hash *doesn't define an ordering* for the sub variables in it. That is, there is no such thing as the first subvariable, and the second subvariable, etc.; the variables are just accessed by name.
- Sequence: Associates an integer number with each of its sub variables. The first subvariable is associated with 0, the second with 1, the third to 2, and so on; the sub variables are ordered. These numbers are often called the *indexes* of the sub variables. Sequences are usually dense, i.e., all indexes up to the index of the last subvariable have an associated subvariable, but it's not strictly necessary. The type of the subvariable values need not be the same.
- Collection: A collection, from the viewpoint of the template author, is a restricted sequence. You cannot access its size or retrieve its sub variables by index, but they can be still listed with the `list` directive.

Note that since a value can have multiple types, it is possible for a value to be both a hash and a sequence, in which case it would support index-based access as well as access by lookup name. However, typically a container will be either a hash or a sequence, not both.

As the value of the variables stored in hashes and sequences (and collections) can be anything, it can be a hash or sequence (or collection) as well. This way you can build arbitrarily deep structures.

The data-model itself (or better said the root of it) is a hash.

Subroutines

Methods and functions

A value that is a method or a function is used to calculate another value, influenced by the parameters you give to it.

For programmer types: Methods/functions are first-class values, just like in functional programming languages. This means that functions/methods can be the parameters or return values of other functions/methods, you can assign them to variables, and so on.

Suppose that programmers have put the method variable `avg` in the data-model that can be used to calculate the average of numbers. If you give the 3 and 5 as parameters when you access `avg`, then you get the value 4.

The usage of methods will be explained later, but perhaps this example helps to understand what methods are:

```
The average of 3 and 5 is: ${avg(3, 5)}
The average of 6 and 10 and 20 is: ${avg(6, 10, 20)}
The average of the price of a python and an elephant is:
${avg(animals.python.price, animals.elephant.price)}
```

this will output:

```
The average of 3 and 5 is: 4
The average of 6 and 10 and 20 is: 12
The average of the price of a python and an elephant is:
4999.5
```

What is the difference between a method and a function? As far as the template author is concerned, nothing. Well not really nothing, as methods typically come from the data-model (*as they reflect the methods of Java objects*), and functions are defined in templates (with the`function` directive -- an advanced topic), but both can be used on the same way.

User-defined directives

A value of this type can be used as user-defined directive (with other words, as FreeMarker tag). An user-defined directive is a subroutine, something like a little reusable template fragment. But this is an advanced topic that will be explained later in its own chapter.

For programmer types: user-defined directives (such as macros), are first-class values too, just like functions/methods are.

Just to get an idea about user-defined directives (so just ignore this if you won't understand), assume we have a variable, `box`, whose value is a user-defined directive that prints some kind of fancy HTML message box with a title bar and a message in it. The `box` variable could be used in the template like this (for example):

```
<@box title="Attention!">
  Too much copy-pasting may leads to
  maintenance headaches.
</@box>
```

Function/method versus user-defined directive

This is for advanced users again (so ignore it if you don't understand). It's a frequent dilemma if you should use a function/method or an user-defined directive to implement something. The rule of thumb is: Implement the facility as user-defined directive instead of as function/method if:

- ... the purpose of it is generating a piece of the output that's not just a single value, and typically involves markup. The template language was designed for printing to the output directly, piece by piece, as it goes though `list` loops, `if`-s, etc. Building up a string value in a variable then returning it is much less convenient.
- ... it's the side-effect that is important and not the return value. For example, a directive whose purpose is to add an entry to the server log is like that. (In fact you can't have a return value for a user-defined directive, but some kind of feedback is still possible by setting non-local variables.)
- ... it will do flow control on the caller side (like for example `list` or `if` directives do). You just can't do that with a function/method.
- ... you are using legacy escaping via the `escape` directive (instead of auto-escaping), and the result contains markup. When you print the result with `${...}`, the markup will be escaped and thus ruined, but if it's printed by a directive call (`<@...>`), it won't be.

The Java methods of FreeMarker-unaware Java objects are normally visible as methods in templates, regardless of the nature of the Java method; you have no choice there.

Miscellaneous

Nodes

Node variables represent a node in a tree structure, and are used mostly with XML processing, which is an advanced, and specialized topic.

Still, a quick overview *for advanced users*: A node is similar to a sequence that stores other nodes, which are often referred as the children nodes. A node stores a reference to its container node, which is often referred as the parent node. The main point of being a node is the topological information; other data must be stored by utilizing that a value can have multiple types. Like, a value may be both a node and a number, in which case it can store a number as the "pay-load". Apart from the topological information, a node can store some metainformation as well: a node name, a node type (string), and a node namespace (string). For example, if the node symbolizes a `h1`element in an XHTML document, then its name could be `"h1"`, it's node type could be `"element"`, and it's namespace could be`"http://www.w3.org/1999/xhtml"`. But it's up to the designer of the data-model if what meaning these metainformations have, and if they are used at all. The way of retrieving the topological and metainformations is described in a later chapter (that you don't have to understand at this point).

Markup output

This type is related to auto-escaping mechanism introduced FreeMarker 2.3.24; you can read about this type there. But in short, this is a value that stores text that's already in the output markup format (like HTML, XML, RTF, etc.), and hence must not be auto-escaped.

Values of this type are usually produced inside the templates (like with `no_esc` built-in or output capturing assignments), but can also be part of the data-model. Such values in the data-model are useful for example if you have message resources that sometimes contain the message in HTML format, rather than in plain text. If the data-model uses HTML markup output values for those messages instead of strings, then the template author need not know which messages contain HTML and which plain text, as double escaping will be avoided automatically when the message is inserted with `${...}`.

Directives

You use FTL tags to call **directives**. In the example you have called the `list` directive. Syntactically you have done it with two tags: `<#list animals as animal>` and `</#list>`.

There are two kind of FTL tags:

- Start-tag: `<#directivename parameters>`
- End-tag: `</#directivename>`

This is similar to HTML or XML syntax, except that the tag name starts with `#`. If the directive doesn't have nested content (content between the start-tag and the end-tag), you must use the start-tag with no end-tag. For example you write `<#if something>...</#if>`, but just`<#include something>` as FreeMarker knows that the `include` directive can't have nested content.

The format of the *parameters* depends on the *directivename*.

In fact there are two types of directives: predefined directives and user-defined directives. For user-defined directives you use `@` instead of `#`, for example `<@mydirective parameters>...</@mydirective>`. Further difference is that if the directive has no nested content, you must use a tag like `<@mydirective parameters />`, similarly as in XML (e.g. `<img ... />`). But user-defined directives is an advanced topic that will be discussed later.

FTL tags, like HTML tags, must be properly nested. So the code below is wrong, as the `if` directive is both inside and outside of the nested content of the `list` directive:

```
<ul>
<#list animals as animal>
  <li>${animal.name} for ${animal.price} Euros
  <#if user == "Big Joe">
     (except for you)
</#list> <#-- WRONG! The "if" has to be closed first. -->
</#if>
</ul>
```

Note that FreeMarker doesn't care about the nesting of HTML tags, only about the nesting of FTL tags. It just sees HTML as flat text, it doesn't interpret it in any way.

If you try to use a non-existing directive (e.g., you mistype the directive name), FreeMarker will decline to use the template and produce an error message.

FreeMarker ignores superfluous white-space inside FTL tags. So you can write this:

```
<#list[BR]
  animals       as[BR]
     animal[BR]
>[BR]
${animal.name} for ${animal.price} Euros[BR]
</#list    >
```

You may not, however, insert white-space between the `<` or `</` and the directive name.

The complete list and description of all directives can be found in the Template Language Reference/Directive Reference (but I recommend that you look at the chapter about expressions first).

Note:

FreeMarker can be configured to use `[` and `]` instead of `<` and `>` in the FTL tags and FTL comments, like `[#if user == "Big Joe"]...[/#if]`. For more information read:Miscellaneous/Alternative (square bracket) syntax.

Note:

FreeMarker can be configured so that it understands predefined directives without `#` (like `<if user == "Big Joe">...</if>`). However we don't recommend the usage of this mode. For more information read: Template Language Reference/Deprecated FTL constructs/Old FTL syntax

Expressions

Page Contents

- Quick overview (cheat sheet)
- Specify values directly
  - Strings
  - Numbers
  - Booleans
  - Sequences
  - Ranges
  - Hashes
- Retrieving variables
  - Top-level variables
  - Retrieving data from a hash
  - Retrieving data from a sequence
  - Special variables
- String operations
  - Interpolation and concatenation
  - Getting a character
  - String slicing (substrings)
- Sequence operations
  - Concatenation
  - Sequence slicing
- Hash operations
  - Concatenation
- Arithmetical calculations
- Comparison
- Logical operations
- Built-ins
- Method call
- Handling missing values
  - Default value operator
  - Missing value test operator
- Assignment Operators
- Parentheses
- White-space in expressions
- Operator precedence

When you supply values for interpolations or directive parameters you can use variables or more complex expressions. For example, if x is the number 8 and y is 5, the value of `(x + y)/2` resolves to the numerical value 6.5.

Before we go into details, let's see some concrete examples:

- When you supply value for interpolations: The usage of interpolations is `${expression}` where expression gives the value you want to insert into the output as text. So `${(5 + 8)/2}` prints ``6.5'' to the output (or possibly ``6,5'' if the language of your output is not US English).
- When you supply a value for the directive parameter: You have already seen the `if` directive in the Getting Started section. The syntax of this directive is: `<#if expression>...</#if>`. The expression here must evaluate to a boolean value. For example in `<#if 2 < 3>` the `2 < 3` (2 is less than 3) is an expression which evaluates to `true`.

Quick overview (cheat sheet)

This is a reminder for those of you who already know FreeMarker or are just experienced programmers:

- Specify values directly
  - Strings: `"Foo"` or `'Foo'` or `"It's \"quoted\""` or `'It\'s "quoted"'` or `r"C:\raw\string"`
  - Numbers: `123.45`
  - Booleans: `true`, `false`
  - Sequences: `["foo", "bar", 123.45]`; Ranges: `0..9`, `0..<10` (or `0..!10`), `0..`
  - Hashes: `{"name":"green mouse", "price":150}`
- Retrieving variables
  - Top-level variables: `user`
  - Retrieving data from a hash: `user.name`, `user["name"]`
  - Retrieving data from a sequence: `products[5]`
  - Special variable: `.main`
- String operations
  - Interpolation and concatenation: `"Hello ${user}!"` (or `"Hello " + user + "!"`)
  - Getting a character: `name[0]`
  - String slice: Inclusive end: `name[0..4]`, Exclusive end: `name[0..<5]`, Length-based (lenient): `name[0..*5]`, Remove starting:`name[5..]`
- Sequence operations
  - Concatenation: `users + ["guest"]`
  - Sequence slice: Inclusive end: `products[20..29]`, Exclusive end: `products[20..<30]`, Length-based (lenient):`products[20..*10]`, Remove starting: `products[20..]`
- Hash operations
  - Concatenation: `passwords + { "joe": "secret42" }`
- Arithmetical calculations: `(x * 1.5 + 10) / 2 - y % 100`
- Comparison: `x == y`, `x != y`, `x < y`, `x > y`, `x >= y`, `x <= y`, `x lt y`, `x lte y`, `x gt y`, `x gte y`, ...etc.
- Logical operations: `!registered && (firstVisit || fromEurope)`
- Built-ins: `name?upper_case`, `path?ensure_starts_with('/')`
- Method call: `repeat("What", 3)`
- Missing value handler operators:
  - Default value: `name!"unknown"` or `(user.name)!"unknown"` or `name!` or `(user.name)!`
  - Missing value test: `name??` or `(user.name)??`
- Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `++`, `--`

See also: Operator precedence

Specify values directly

Often you want to specify a value directly and not as a result of some calculations.

Strings

To specify a string value directly you give the text in quotation marks, e.g.: `"some text"` or in apostrophe-quote, e.g. `'some text'`. The two forms are equivalent. If the text itself contains the character used for the quoting (either `"` or `'`) or backslashes, you have to precede them with a backslash; this is called escaping. You can type any other character, including line breaks, in the text directly. Example:

```
${"It's \"quoted\" and
this is a backslash: \\"}

${'It\'s "quoted" and
this is a backslash: \\'}
```

will print:

```
It's "quoted" and
this is a backslash: \

It's "quoted" and
this is a backslash: \
```

Note:

Of course, you could simply type the above text into the template, without using `${...}`. But we do it here just for the sake of example, to demonstrate expressions.

This is the list of all supported escape sequences. All other usage of backlash in string literals is an error and any attempt to use the template will fail.

Escape sequence	Meaning
`\"`	Quotation mark (u0022)
`\'`	Apostrophe (a.k.a. apostrophe-quote) (u0027)
`\{`	Opening curly brace: `{`
`\\`	Back slash (u005C)
`\n`	Line feed (u000A)
`\r`	Carriage return (u000D)
`\t`	Horizontal tabulation (a.k.a. tab) (u0009)
`\b`	Backspace (u0008)
`\f`	Form feed (u000C)
`\l`	Less-than sign: `<`

Escape sequence	Meaning
`\g`	Greater-than sign: `>`
`\a`	Ampersand: `&`
`\x`*`Code`*	Character given with its hexadecimal Unicode code (UCS code)

The *`Code`* after the `\x` is 1 to 4 hexadecimal digits. For example this all put a copyright sign into the string: `"\xA9 1999-2001"`,`"\x0A9 1999-2001"`, `"\x00A9 1999-2001"`. When the character directly after the last hexadecimal digit can be interpreted as hexadecimal digit, you must use all 4 digits or else FreeMarker will misunderstand you.

Note that the character sequence `${` (and `#{`) has special meaning. It's used to insert the value of expressions (typically: the value of variables, as in `"Hello ${user}!"`). This will be explained later. If you want to print `${` or `#{`, you should either use raw string literals as explained below, or escape the `{` like in `"foo $\{bar}"`.

A special kind of string literals is the raw string literals. In raw string literals, backslash and `${` have no special meaning, they are considered as plain characters. To indicate that a string literal is a raw string literal, you have to put an `r` directly before the opening quotation mark or apostrophe-quote. Example:

```
${r"${foo}"}
${r"C:\foo\bar"}
```

will print:

```
${foo}
C:\foo\bar
```

Numbers

To specify a numerical value directly you type the number without quotation marks. You have to use the dot as your decimal separator and must not use any grouping separator symbols. You can use `-` or `+` to indicate the sign (`+` is redundant). Scientific notation is not yet supported (so `1E3` is wrong). Also, you cannot omit the 0 before the decimal separator (so `.5` is wrong).

Examples of valid number literals: `0.08`, `-5.013`, `8`, `008`, `11`, `+11`

Note that numerical literals like `08`, `+8`, `8.00` and `8` are totally equivalent as they all symbolize the number eight. Thus, `${08}`, `${+8}`,`${8.00}` and `${8}` will all print exactly same.

Booleans

To specify a boolean value you write `true` or `false`. Don't use quotation marks.

Sequences

To specify a literal sequence, you list the sub variables separated by commas, and put the whole list into square brackets. For example:

```
<#list ["foo", "bar", "baz"] as x>
${x}
</#list>
```

will print:

```
foo
bar
baz
```

The items in the list are expressions, so you can do this for example: `[2 + 2, [1, 2, 3, 4], "foo"]`. Here the first subvariable will be the number 4, the second will be another sequence, and the third subvariable will be the string "foo".

Ranges

Ranges are just sequences, but they are created by specifying what range of whole numbers they contain, instead of specifying their items one by one. For example, `0..<m`, assuming the `m` variable stores 5, will give a sequence that contains `[0, 1, 2, 3, 4]`. Ranges are primarily used for iterating over a range of numbers with `<#list ...>` and for slicing sequences and slicing strings.

The generic forms of range expressions are (where *`start`* and *`end`* can be any expression that evaluates to a number):

- *`start`*`..`*`end`*: Range with inclusive end. For example, `1..4` gives `[1, 2, 3, 4]`, and `4..1` gives `[4, 3, 2, 1]`. Beware, ranges with inclusive end never give an empty sequence, so `0..length-1` is *WRONG*, because when length is `0` it gives `[0, -1]`.
- *`start`*`..<`*`end`* or *`start`*`..!`*`end`*: Range with exclusive end. For example, `1..<4` gives `[1, 2, 3]`, `4..<1` gives `[4, 3, 2]`, and `1..<1` gives `[]`. Note the last example; the result can be an empty sequence. There's no difference between `..<` and `..!`; the last form is used in applications where using the `<` character causes problems (for HTML editors and such).
- *`start`*`..*`*`length`*: Length limited range. For example, `10..*4` gives `[10, 11, 12, 13]`, `10..*-4` gives `[10, 9, 8, 7]`, and`10..*0` gives `[]`. When these kind of ranges are used for slicing, the slice will end without error if the end of the sliced sequence or string is reached before the specified range length was reached; see slicing sequences for more.

  Note:

  Length limited ranges were introduced in FreeMarker 2.3.21.

- *`start`*`..`: Right-unbounded range. This are like length limited ranges with infinite length. For example `1..` gives `[1, 2, 3, 4, 5, 6, ... ]`, up to infinity. Be careful when processing (like listing) such ranges, as processing all items of it it would take forever or until the application runs out of memory and crashes. Just like with length limited ranges, when these kind of ranges are used for slicing, the slice will end when the end of the sliced sequence or string is reached.

Warning!

Right-unbounded ranges before FreeMarker 2.3.21 were only used for slicing, and behaved like an empty sequence for other purposes. To activate the new behavior, it's not enough to use FreeMarker 2.3.21, the programmer also have to set the `incompatible_improvements`configuration setting to at least 2.3.21.

Further notes on ranges:

- Range expressions themselves don't have square brackets, for example, you write `<#assign myRange = 0..<x`, NOT `<#assign myRange = [0..<x]>`. The last would create a sequence that contains an item that's a range. The square brackets are part of the slicing syntax, like *`seq[myRange]`*.
- You can write arithmetical expression on the sides of the `..` without parenthesis, like `n + 1 ..< m / 2 - 1`.
- `..`, `..<`, `..!` and `..*` are operators, so you can't have space inside them. Like `n .. <m` is WRONG, but `n ..< m` is good.
- The reported size of right-unbounded ranges is 2147483647 (or 0 if `incompatible_improvements` is less than 2.3.21) due to a technical limitation (32 bits). However, when listing them, their actual size is infinite.
- Ranges don't really store the numbers they consist of, thus for example `0..1` and `0..100000000` is equally fast to create and takes the same amount of memory.

Hashes

To specify a hash in a template, you list the key/value pairs separated by commas, and put the list into curly brackets. The key and value within a key/value pair are separated with a colon. Here is an example: `{ "name": "green mouse", "price": 150 }`. Note that both the names and the values are expressions. The keys must be strings. The values can be if any type.

Retrieving variables

Top-level variables

To access a top-level variable, you simply use the variable name. For example, the expression `user` will evaluate to the value of variable stored with name "user" in the root. So this will print what you store there:

```
${user}
```

If there is no such top-level variable, then an error will result when FreeMarker tries to evaluate the expression, and it aborts template processing (unless programmers has configured FreeMarker differently).

In this kind of expression, the variable name can only contain letters (including non-Latin letters), digits (including non-Latin digits), underline (`_`), dollar (`$`), at sign (`@`). Furthermore, the first character can't be a ASCII digit (`0`-`9`). Starting from FreeMarker 2.3.22, the variable name can also contain minus (`-`), dot (`.`), and colon (`:`) at any position, but these must be escaped with a preceding backslash (`\`), or else they would be interpreted as operators. For example, to read the variable whose name is "data-id", the expression is `data\-id`, as `data-id`would be interpreted as "data minus id". (Note that these escapes only work in identifiers, not in string literals.)

Retrieving data from a hash

If we already have a hash as a result of an expression, then we can get its subvariable with a dot and the name of the subvariable. Assume that we have this data-model:

```
(root)
 |
 +- book
 |    |
 |    +- title = "Breeding green mouses"
 |    |
 |    +- author
 |         |
 |         +- name = "Julia Smith"
 |         |
 |         +- info = "Biologist, 1923-1985, Canada"
 |
 +- test = "title"
```

Now we can read the `title` with `book.title`, since the book expression will return a hash (as explained in the last chapter). Applying this logic further, we can read the name of the author with this expression: `book.author.name`.

There is an alternative syntax if we want to specify the subvariable name with an expression: `book["title"]`. In the square brackets you can give any expression as long as it evaluates to a string. So with this data-model you can also read the title with `book[test]`. More examples; these are all equivalent: `book.author.name`, `book["author"].name`, `book.author.["name"]`, `book["author"]["name"]`.

When you use the dot syntax, the same restrictions apply regarding the variable name as with top-level variables (name can contain only letters, digits, `_`, `$`, `@` but can't start with `0`-`9`, also starting from 2.3.22 you can also use `\-`, `\.` and `\:`). There are no such restrictions when you use the square bracket syntax, since the name is the result of an arbitrary expression. (Note, that to help the FreeMarker XML support, if the subvariable name is `*` (asterisk) or `**`, then you do not have to use square bracket syntax.)

As with the top-level variables, trying to access a non-existent subvariable causes an error and aborts the processing of the template (unless programmers has configured FreeMarker differently).

Retrieving data from a sequence

This is the same as for hashes, but you can use the square bracket syntax only, and the expression in the brackets must evaluate to a number, not a string. For example to get the name of the first animal of the example data-model (remember that the number of the first item is 0, not 1): `animals[0].name`

Special variables

Special variables are variables defined by the FreeMarker engine itself. To access them, you use the `.variable_name` syntax.

Normally you don't need to use special variables. They are for expert users. The complete list of special variables can be found in thereference.

String operations

Interpolation and concatenation

If you want to insert the value of an expression into a string, you can use `${...}` (and the deprecated `#{...}`) in string literals. `${...}` in string literals behaves similarly as in text sections (so it goes through the same *locale sensitive* number and date/time formatting).

Example (assume that user is "Big Joe"):

```
<#assign s = "Hello ${user}!">
${s} <#-- Just to see what the value of s is -->
```

This will print:

```
Hello Big Joe!
```

Warning!

A frequent mistake of users is the usage of interpolations in places where they needn't/shouldn't/can't be used. Interpolations work *only* in text sections (e.g. `<h1>Hello ${name}!</h1>`) and in string literals (e.g. `<#include "/footer/${company}.html">`). A typical *WRONG* usage is `<#if ${big}>...</#if>`, which will cause a syntactical error. You should simply write `<#if big>...</#if>`. Also, `<#if "${big}">...</#if>` is *WRONG*, since it converts the parameter value to string and the `if` directive wants a boolean value, so it will cause a runtime error.

Alternatively, you can use the `+` operator to achieve similar result:

```
<#assign s = "Hello " + user + "!">
```

This gives the same result as the earlier example with the `${...}`.

Warning!

Because `+` follows similar rules as `${...}`, the appended string is influenced by the `locale`,`number_format`, `date_format`, `time_format`, `datetime_format` and `boolean_format`, etc. settings, and thus the result targets humans and isn't in generally machine parseable. This mostly leads to problems with numbers, as many locales use grouping (thousands separators) by default, and so `"someUrl?id=" + id` becomes to something like `"someUrl?id=1 234"`. To prevent this, use the `?c` (for Computer audience) built-in, like in `"someUrl?id=" + id?c` or`"someUrl?id=${id?c}"`, which will evaluate to something like `"someUrl?id=1234"`, regardless of locale and format settings.

As when `${...}` is used inside string *expressions* it's just a shorthand of using the `+` operator, auto-escaping is not applied on it.

Getting a character

You can get a single character of a string at a given index similarly as you can read the subvariable of a sequence, e.g. `user[0]`. The result will be a string whose length is 1; FTL doesn't have a separate character type. As with sequence sub variables, the index must be a number that is at least 0 and less than the length of the string, or else an error will abort the template processing.

Since the sequence subvariable syntax and the character getter syntax clashes, you can use the character getter syntax only if the variable is not a sequence as well (which is possible because FTL supports multi-typed values), since in that case the sequence behavior prevails. (To work this around, you can use the `string` built-in, e.g. `user?string[0]`. Don't worry if you don't understand this yet; built-ins will be discussed later.)

Example (assume that user is "Big Joe"):

```
${user[0]}
${user[4]}
```

will print (note that the index of the first character is 0):

```
B
J
```

String slicing (substrings)

You can a slice a string in the same way as you slice a sequence (see there), only here instead of sequence items you work with characters. Some differences are:

- Decreasing ranges aren't allowed for string slicing. (That's because unlike sequences, you seldom if ever want to show a string reversed, so if that happens, that's almost always the result of an oversight.)
- If a value is both a string and a sequence (a multi-typed value), then slicing will slice the sequence instead of the string. When you are processing XML, such values are common. In such cases you can use *`someXMLnode`*`?string[`*`range`*`]`.
- There's a legacy bug where a range with *inclusive* end that's one less than the starting index and is non-negative (like in `"abc"[1..0]`) will give an empty string instead of an error. (It should be an error as it's a decreasing range.) Currently this bug is emulated for backward compatibility, but you shouldn't utilize it, as in the future it will be certainly an error.

Example:

```
<#assign s = "ABCDEF">
${s[2..3]}
${s[2..<4]}
${s[2..*3]}
${s[2..*100]}
${s[2..]}
```

will print:

```
CD
CD
CDE
CDEF
CDEF
```

Note:

Some of the typical use-cases of string slicing is covered by convenient built-ins:`remove_beginning`, `remove_ending`, `keep_before`, `keep_after`, `keep_before_last`,`keep_after_last`

Sequence operations

Concatenation

You can concatenate sequences in the same way as strings, with `+`. Example:

```
<#list ["Joe", "Fred"] + ["Julia", "Kate"] as user>
- ${user}
</#list>
```

will print:

```
- Joe
- Fred
- Julia
- Kate
```

Note that sequence concatenation is not to be used for many repeated concatenations, like for appending items to a sequence inside a loop. It's just for things like `<#list users + admins as person>`. Although concatenating sequences is fast and its speed is independently of the size of the concatenated sequences, the resulting sequence will be always a little bit slower to read than the original two sequences were. This way the result of many repeated concatenations is a sequence that is slow to read.

Sequence slicing

With *`seq[range]`*, were *`range`* is a range value as described here, you can take a slice of the sequence. The resulting sequence will contain the items from the original sequence (*`seq`*) whose indexes are in the range. For example:

```
<#assert seq = ["A", "B", "C", "D", "E"]>
<#list seq[1..3] as i>${i}</#list>
```

will print

```
BCD
```

Furthermore, the items in the slice will be in the same order as in the range. Thus for example the above example with the `3..1` range would print `DCB`.

The numbers in the range must be valid indexes in the sequence, or else the processing of the template will be aborted with error. Like in the last example, `seq[-1..0]` would be an error as `seq[-1]` is invalid, also `seq[1..5]` would be because `seq[5]` is invalid. (Note that`seq[100..<100]` or `seq[100..*0]` would be valid despite that 100 is out of bounds, because those ranges are empty.)

Length limited ranges (*`start..*length`*) and right-unbounded ranges (*`start..`*) adapt to the length of the sliced sequence. They will slice out at most as many items as there is available:

```
<#assign seq = ["A", "B", "C"]>

Slicing with length limited ranges:
- <#list seq[0..*2] as i>${i}</#list>
```

```
- <#list seq[1..*2] as i>${i}</#list>
- <#list seq[2..*2] as i>${i}</#list> <#-- Not an error -->
- <#list seq[3..*2] as i>${i}</#list> <#-- Not an error -->

Slicing with right-unlimited ranges:
- <#list seq[0..] as i>${i}</#list>
- <#list seq[1..] as i>${i}</#list>
- <#list seq[2..] as i>${i}</#list>
- <#list seq[3..] as i>${i}</#list>
```

This will print:

```
Slicing with length limited ranges:
- AB
- BC
- C
-

Slicing with right-unlimited ranges:
- ABC
- BC
- C
-
```

Note above that slicing with length limited and right unbounded ranges allow the starting index to be past the last item *by one* (but no more).

Note:

To split a sequence to slices of a given size, you should use the `chunk` built-in.

Hash operations

Concatenation

You can concatenate hashes in the same way as strings, with `+`. If both hashes contain the same key, the hash on the right-hand side of the`+` takes precedence. Example:

```
<#assign ages = {"Joe":23, "Fred":25} + {"Joe":30, "Julia":18}>
- Joe is ${ages.Joe}
- Fred is ${ages.Fred}
- Julia is ${ages.Julia}
```

will print:

```
- Joe is 30
- Fred is 25
- Julia is 18
```

Note that hash concatenation is not to be used for many repeated concatenations, like for adding items to a hash inside a loop. It's the same as with the sequence concatenation.

Arithmetical calculations

This is the basic 4-function calculator arithmetic plus the modulus operator. So the operators are:

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*`
- Division: `/`
- Modulus (remainder): `%`

Example:

```
${100 - x * x}
${x / 2}
${12 % 10}
```

Assuming that `x` is 5, it will print:

```
75
2.5
2
```

Both operands must be expressions which evaluate to a numerical value. So the example below will cause an error when FreeMarker tries to evaluate it, since `"5"` is a string and not the number 5:

```
${3 * "5"} <#-- WRONG! -->
```

There is an exception to the above rule. The `+` operator, is used to concatenate strings as well. If on one side of `+` is a string and on the other side of `+` is a numerical value, then it will convert the numerical value to string (using the format appropriate for language of the page) and then use the `+` as string concatenation operator. Example:

```
${3 + "5"}
```

will output this:

```
35
```

Generally, FreeMarker never converts a string to a number automatically, but it may convert a number to a string automatically.

People often want only the integer part of the result of a division (or of other calculations). This is possible with the `int` built-in. (Built-ins are explained later):

```
${(x/2)?int}
${1.1?int}
${1.999?int}
${-1.1?int}
${-1.999?int}
```

Assuming that `x` is 5, it will print:

```
2
1
1
-1
-1
```

Comparison

Sometimes you want to know if two values are equal or not, or which value is the greater.

To show concrete examples I will use the `if` directive here. The usage of `if` directive is: `<#if expression>...</#if>`, where expression must evaluate to a boolean value or else an error will abort the processing of the template. If the value of expression is `true` then the things between the begin and end-tag will be processed, otherwise they will be skipped.

To test two values for equality you use `==` (or `=` as a *deprecated* alternative) To test two values for inequality you use `!=`. For example, assume that `user` is ``Big Joe'':

```
<#if user == "Big Joe">
  It is Big Joe
</#if>
<#if user != "Big Joe">
  It is not Big Joe
</#if>
```

The `user == "Big Joe"` expression in the `<#if ...>` will evaluate to the boolean `true`, so the above will say ``It is Big Joe''.

The expressions on both sides of the `==` or `!=` must evaluate to a scalar (not a sequence or hash). Furthermore, the two scalars must have the same type (i.e. strings can only be compared to strings and numbers can only be compared to numbers, etc.) or else an error will abort template processing. For example `<#if 1 == "1">` will cause an error. Note that FreeMarker does exact comparison, so string comparisons are case and white-space sensitive: `"x"` and `"x "` and `"X"` are not equal values.

For numerical and date, time and date-time values you can also use `<`, `<=`, `>=` and `>`. You can't use them for strings! Example:

```
<#if x <= 12>
  x is less or equivalent with 12
</#if>
```

There's a problem with `>=` and `>`. FreeMarker interprets the `>` character as the closing character of the FTL tag. To prevent this, you can use`lt` instead of `<`, `lte` instead of `<=`, `gt` instead of `>` and `gte` instead of `>=`, like in `<#if x gt y>`. Another trick it to put the expression into parentheses like in `<#if (x > y)>`, although it's considered to be less elegant.

Note:

FreeMarker supports some other alternatives too, but these are deprecated:

- Writing `>` and `<` on the place of the problematic relation marks, like in: `<#if x > y>` or `<#if x >= y>` . Note that in general FTL does not support entity references (the `&...;` things) in FTL tags; it's just an exception with the arithmetical comparisons.
- `\lt`, `\lte`, `\gt` and `\gte` which are the same as the ones without the backslash

Logical operations

Just the usual logical operators:

- Logical or: `||`
- Logical and: `&&`
- Logical not: `!`

The operators will work with boolean values only. Otherwise an error will abort the template processing.

Example:

```
<#if x < 12 && color == "green">
  We have less than 12 things, and they are green.
</#if>
<#if !hot> <#-- here hot must be a boolean -->
  It's not hot.
</#if>
```

Built-ins

Built-ins are like methods that are added to the objects by FreeMarker. To prevent name clashes with actual methods and other sub-variables, instead of dot (`.`), you separate them from the parent object with question mark (`?`). For example, if you want to ensure that `path` has an initial `/` then you could write `path?ensure_starts_with('/')`. The Java object behind `path` (a `String` most certainly) doesn't have such method, FreeMarker adds it. For brevity, if the method has no parameters, you *must* omit the `()`, like, to get the length of `path`, you have to write `path?length`, *not* `path?length()`.

The other reason why built-ins are crucial is that normally (though it depends on configuration settings), FreeMarker doesn't expose the Java API of the objects. So despite that Java's `String` has a `length()` method, it's hidden from the template, you *have to* use `path?length`instead. The advantage of that is that thus the template doesn't depend on the exactly type of the underlying Java objects. (Like `path` is maybe a `java.nio.Path` behind the scenes, but if the programmers has configure FreeMarker to expose `Path` objects as FTL strings, the template won't be aware of that, and `?length` will work, despite that `java.nio.Path` has no similar method.)

You can find some of the most commonly used built-ins mentioned here, and the complete list of built-ins in the Reference. For now, just a few of the more important ones:

Example:

```
${testString?upper_case}
${testString?html}
${testString?upper_case?html}

${testSequence?size}
${testSequence?join(", ")}
```

Assuming that `testString` stores the string ``Tom & Jerry'', and testSequnce stores the strings "foo", "bar" and "baz", the output will be:

```
TOM & JERRY
Tom &amp;amp; Jerry
TOM &amp;amp; JERRY
```

```
3
foo, bar, baz
```

Note the `test?upper_case?html` above. Since the result of `test?upper_case` is a string, you can apply the `html` built-in on it.

Naturally, the left side of the built-in can be arbitrary expression, not just a variable name:

```
${testSeqence[1]?cap_first}
${"horse"?cap_first}
${(testString + " & Duck")?html}
Bar
Horse
Tom &amp;amp; Jerry &amp;amp; Duck
```

Method call

If you have a method then you can use the method call operation on it. The method call operation is a comma-separated list of expressions in parentheses. These values are called parameters. The method call operation passes these values to the method which will in turn return a result. This result will be the value of the whole method call expression.

For example, assume the programmers have made available a method variable called `repeat`. You give a string as the first parameter, and a number as the second parameter, and it returns a string which repeats the first parameter the number of times specified by the second parameter.

```
${repeat("Foo", 3)}
```

will print:

```
FooFooFoo
```

Here `repeat` was evaluated to the method variable (according to how you access top-level variables) and then `("What", 3)` invoked that method.

I would like to emphasize that method calls are just plain expressions, like everything else. So this:

```
${repeat(repeat("x", 2), 3) + repeat("Foo", 4)?upper_case}
```

will print this:

```
xxxxxxFOOFOOFOOFOO
```

Handling missing values

Note:

These operators exist since FreeMarker 2.3.7 (replacing the `default`, `exists` and `if_exists`built-ins).

As we explained earlier, an error will occur and abort the template processing if you try to access a missing variable. However two special operators can suppress this error, and handle the

problematic situation. The handled variable can be top-level variable, hash subvariable, or sequence subvariable as well. Furthermore these operators handle the situation when a method call doesn't return a value *(from the viewpoint of Java programmers: it returns `null` or it's return type is `void`)*, so it's more correct to say that these operators handle missing values in general, rather than just missing variables.

For those who know what's Java `null`, FreeMarker 2.3.x treats them as missing values. Simply, the template language doesn't know the concept of `null`. For example, if you have a bean that has a `maidenName` property, and the value of that property is `null`, then that's the same as if there were no such property at all, as far as the template is concerned (assuming you didn't configured FreeMarker to use some extreme object wrapper, that is). The result of a method call that returns `null` is also treated as a missing variable (again, assuming that you use some usual object wrapper). See more in the FAQ.

Note:

If you wonder why is FreeMarker so picky about missing variables, read this FAQ entry.

Default value operator

Synopsis: *`unsafe_expr`*`!`*`default_expr`* or *`unsafe_expr`*`!` or `(`*`unsafe_expr`*`)!`*`default_expr`* or `(`*`unsafe_expr`*`)!`

This operator allows you to specify a default value for the case when the value is missing.

Example. Assume no variable called `mouse` is present:

```
${mouse!"No mouse."}
<#assign mouse="Jerry">
${mouse!"No mouse."}
```

The output will be:

```
No mouse.
Jerry
```

The default value can be any kind of expression, so it doesn't have to be a string. For example you can write `hits!0` or `colors!["red", "green", "blue"]`. There is no restriction regarding the complexity of the expression that specifies the default value, for example you can write: `cargo.weight!(item.weight * itemCount + 10)`.

Warning!

If you have a composite expression after the `!`, like `1 + x`, *always* use parentheses, like `${x!(1 + y)}` or `${(x!1) + y)}`, depending on which interpretation you meant. That's needed because due to a programming mistake in FreeMarker 2.3.x, the precedence of `!` (when it's used as default value operator) is very low at its right side. This means that, for example, `${x!1 + y}`is misinterpreted by FreeMarker as `${x!(1 + y)}` while it should mean `${(x!1) + y}`. This programming error will be fixed in FreeMarker 2.4, so you should not utilize this wrong behavior, or else your templates will break with FreeMarker 2.4!

If the default value is omitted, then it will be empty string and empty sequence and empty hash at the same time. (This is possible because FreeMarker allows multi-type values.) Note the consequence that you can't omit the default value if you want it to be `0` or `false`. Example:

```
(${mouse!})
<#assign mouse = "Jerry">
(${mouse!})
```

The output will be:

```
()
(Jerry)
```

Warning!

Due to syntactical ambiguities `<@something a=x! b=y />` will be interpreted as `<@something a=x!(b=y) />`, that is, the `b=y` will be interpreted as a comparison that gives the default value for `x`, rather than the specification of the `b` parameter. To prevent this, write: `<@something a=(x!) b=y />`

You can use this operator in two ways with non-top-level variables:

```
product.color!"red"
```

This will handle if `color` is missing inside `product` (and returns `"red"` if so), but will not handle if `product` is missing. That is, the`product` variable itself must exist, otherwise the template processing will die with error.

```
(product.color)!"red"
```

This will handle if `product.color` is missing. That is, if `product` is missing, or `product` exists but it does not contain `color`, the result will be `"red"`, and no error will occur. The important difference between this and the previous example is that when surrounded with parentheses, it is allowed for any component of the expression to be undefined, while without parentheses only the last component of the expression is allowed to be undefined.

Of course, the default value operator can be used with sequence sub variables as well:

```
<#assign seq = ['a', 'b']>
${seq[0]!'-'}
${seq[1]!'-'}
${seq[2]!'-'}
${seq[3]!'-'}
```

the outpur will be:

```
a
b
-
-
```

A negative sequence index (as `seq[-1]!'-'`) will always cause an error, you can't suppress that with this or any other operator.

Missing value test operator

Synopsis: *`unsafe_expr`*`??` or `(`*`unsafe_expr`*`)??`

This operator tells if a value is missing or not. Depending on that, the result is either `true` or `false`.

Example. Assume no variable called `mouse` is present:

```
<#if mouse??>
  Mouse found
<#else>
  No mouse found
</#if>
Creating mouse...
<#assign mouse = "Jerry">
<#if mouse??>
  Mouse found
<#else>
  No mouse found
</#if>
```

The output will be:

```
  No mouse found
Creating mouse...
  Mouse found
```

With non-top-level variables the rules are the same as with the default value operator, that is, you can write `product.color??` and `(product.color)??`.

Assignment Operators

These are actually not expressions, but parts of the syntax of the assignment directives, such as `assign`, `local` and `global`. As such, they can't be used anywhere else.

`<#assign x += y>` is shorthand for `<#assign x = x + y>`, `<#assign x *= y>` is shorthand for `<#assign x = x * y>`, and so on.

`<#assign x++>` differs from `<#assign x += 1>` (or `<#assign x = x + 1>`) in that it always does arithmetical addition (and fails if the variable is not a number), while the others are overloaded to do string and sequence concatenation and hash addition. `<#assign x-->` is shorthand for `<#assign x -= 1>`.

Parentheses

Parentheses can be used to group any expressions. Some examples:

```
                                <#-- Output will be: -->
${3 * 2 + 2}                    <#-- 8 -->
${3 * (2 + 2)}                  <#-- 12 -->
${3 * ((2 + 2) * (1 / 2))}      <#-- 6 -->
${"green " + "mouse"?upper_case}    <#-- green MOUSE -->
${("green " + "mouse")?upper_case}  <#-- GREEN MOUSE -->
<#if !(color == "red" || color == "green")>
  The color is nor red nor green
</#if>
```

Note that the parentheses of a method call expressions have nothing to do with the parentheses used for grouping.

White-space in expressions

FTL ignores superfluous white-space in expressions. So these are totally equivalent:

```
${x + ":" + book.title?upper_case}
```

and

```
${x+":"+book.title?upper_case}
```

and

```
${
   x
 + ":"   +  book   .   title
   ?   upper_case
      }
```

Operator precedence

The following table shows the precedence assigned to the operators. The operators in this table are listed in precedence order: the higher in the table an operator appears, the higher its precedence. Operators with higher precedence are evaluated before operators with a relatively lower precedence. Operators on the same line have equal precedence. When binary operators (operators with two ``parameters'', as + and –) of equal precedence appear next to each other, they are evaluated in left-to-right order.

Operator group	Operators
highest precedence operators	*[subvarName]* *[subStringRange]* . ? *(methodParams)* *expr*! *exp*
unary prefix operators	+*expr* -*expr* !expr
multiplicative operators	* / %
additive operators	+ -
numerical ranges	.. ..< ..! ..*
relational operators	< > <= >= (and equivalents: gt, lt, etc.)
equality operators	== != (and equivalents: =)
logical "and" operator	&&
logical "or" operator	||

For those of you who know C, Java language or JavaScript, note that the precedence rules are the same as in those languages, except that FTL has some operators that do not exist in those languages.

The default value operator (*exp*`!`*exp*) is not yet in the table because of a programming mistake, which will be only fixed in FreeMarker 2.4 due to backward compatibility constraints. It meant to be a "highest precedence operator", but in FreeMarker 2.3.x the precedence on its right side is very low by accident. So if you have a composite expression on the right side, always use paranthesses, etiher like `x!(y + 1)` or like `(x!y) + 1`. Never write just `x!y + 1`.

Defining your own directives

Page Contents

- Basics
- Parameters
- Nested content
- Macros with loop variables
- More about user-defined directives and macros

As far as template authors are concerned, user-defined directives can be defined using the `macro` directive. *Java programmers who want to implement directives in Java Language, rather than in a template, should use `freemarker.template.TemplateDirectiveModel` (seemore here...).*

Basics

A macro is a template fragment associated with a variable. You can use that variable in your template as a user-defined directive, so it helps in repetitive tasks. For example, this creates a macro variable that prints a big ``Hello Joe!'':

```
<#macro greet>
  <font size="+2">Hello Joe!</font>
</#macro>
```

The `macro` directive itself does not print anything; it just creates the macro variable, so there will be a variable called `greet`. Things between the `<#macro greet>` and `</#macro>` (called **macro definition body**) will be executed only when you use the variable as directive. You use user-defined directives by writing `@` instead of `#` in the FTL tag. Use the variable name as the directive name. Also, the end-tag for user-defined directives is mandatory. So you use `greet` like this:

```
<@greet></@greet>
```

But since `<anything></anything>` is equivalent with `<anything/>` you should use this shorter form (that is familiar for you if you knowXML):

```
<@greet/>
```

This will print:

```
  <font size="+2">Hello Joe!</font>
```

But macros can do much more, since the thing between `<#macro ...>` and `</#macro>` is a template fragment, thus it can contain interpolations (`${...}`) and FTL tags (e.g. `<#if ...>...</#if>`).

Note:

Programmers will say on `<@...>` that you **call** the macro.

Parameters

Let's improve the `greet` macro so it can use arbitrary name, not only ``Joe''. For this purpose you can use **parameters**. You define the parameters after the name of the macro in the `macro` directive. Here we define one parameter for the `greet` macro, `person`:

```
<#macro greet person>
  <font size="+2">Hello ${person}!</font>
</#macro>
```

and then you can use this macro as:

```
<@greet person="Fred"/> and <@greet person="Batman"/>
```

which is similar to HTML syntax. This will print:

```
  <font size="+2">Hello Fred!</font>
 and   <font size="+2">Hello Batman!</font>
```

As you can see, the actual value of the macro parameter is accessible in the macro definition body as a variable (`person`). As withpredefined directives, the value of a parameter (the right side of `=`) is an FTL expression. Thus, unlike with HTML, the quotation marks around`"Fred"` and `"Batman"` are not optional. `<@greet person=Fred/>` would mean that you use the value of variable `Fred` for the `person`parameter, rather than the string `"Fred"`. Of course parameter value need not be a string, it can be number, boolean, hash, sequence, etc., also you can use complex expression on the right side of `=` (e.g. `someParam=(price + 50)*1.25`).

User-defined directives can have multiple parameters. For example, add a new parameter `color`:

```
<#macro greet person color>
  <font size="+2" color="${color}">Hello ${person}!</font>
</#macro>
```

and then you can use this macro like:

```
<@greet person="Fred" color="black"/>
```

The order of parameters is not important, so this is equivalent with the previous:

```
<@greet color="black" person="Fred"/>
```

When you call the macro, you can use only parameters that you have defined in the `macro` directive (in this case: `person` and `color`). So if you try `<@greet person="Fred" color="black" background="green"/>` then you will get an error, since you haven't mentioned parameter `background` in the `<#macro ...>`.

Also, you must give value for all parameters that you have defined for the macro. So if you try `<@greet person="Fred"/>` then you will get an error, since you forgot to specify the value of `color`. However, it often happens that you would specify the same value for a parameter in most cases, so you want to specify the value only when you want a different value for it than the usual. This can be achieved if you specify the parameter in the `macro` directive as *`param_name=usual_value`*. For example, you want to use `"black"` for `color` if you don't specify value for that parameter when you use the `greet` directive:

```
<#macro greet person color="black">
  <font size="+2" color="${color}">Hello ${person}!</font>
</#macro>
```

Now `<@greet person="Fred"/>` is OK, since it is equivalent with `<@greet person="Fred" color="black"/>`, thus the value of `color`parameter is known. If you want `"red"` for `color`, then you write `<@greet person="Fred" color="red"/>`, and this value will override the usual value specified with the `macro` directive, so the value of `color` parameter will be `"red"`.

Also, it is important to realize that -- according to the already explained FTL expression rules -- `someParam=foo` and `someParam="${foo}"`are very different. In the fist case, you use the value of variable `foo` as the value of the parameter. In the second case, you use a string literal with interpolation, so the value of the parameter will be a string -- in this case, the value of `foo` rendered to text -- regardless of the type (as number, date, etc.) of `foo`. Or, another example: `someParam=3/4` and `someParam="${3/4}"` are different. If the directive wants a numerical value for `someParam`, it will not like the second variation. Do not exchange these.

A very important aspect of macro parameters is that they are local variables. For more information about local variables please read: Defining variables in the template

Nested content

Custom directive can have nested content, similarly as predefined directives like `<#if ...>nested content</#if>` can have. For example, this creates a macro that draws borders around its nested content:

```
<#macro border>
  <table border=4 cellspacing=0 cellpadding=4><tr><td>
    <#nested>
  </tr></td></table>
</#macro>
```

The `<#nested>` directive executes the template fragment between the start-tag and end-tags of the directive. So if you do this:

```
<@border>The bordered text</@border>
```

the output will be:

```
  <table border=4 cellspacing=0 cellpadding=4><tr><td>
    The bordered text
  </td></tr></table>
```

The `nested` directive can be called for multiple times, for example:

```
<#macro do_thrice>
  <#nested>
  <#nested>
  <#nested>
</#macro>
<@do_thrice>
  Anything.
</@do_thrice>
```

will print:

```
  Anything.
  Anything.
  Anything.
```

If you don't use the `nested` directive, then the nested content will not be executed. Thus, if you accidentally use the `greet` directive like this:

```
<@greet person="Joe">
  Anything.
</@greet>
```

then FreeMarker will not see this as an error, and simply prints:

```
<font size="+2">Hello Joe!</font>
```

and the nested content will be ignored, since the `greet` macro never uses `nested` directive.

The nested content can be anything that is valid FTL, including other user-defined directives. Thus this is OK:

```
<@border>
  <ul>
  <@do_thrice>
    <li><@greet person="Joe"/>
  </@do_thrice>
  </ul>
</@border>
```

and will print:

```
  <table border=4 cellspacing=0 cellpadding=4><tr><td>
      <ul>
    <li><font size="+2">Hello Joe!</font>

    <li><font size="+2">Hello Joe!</font>

    <li><font size="+2">Hello Joe!</font>

  </ul>

  </tr></td></table>
```

The local variables of a macro are not visible in the nested content. Say, this:

```
<#macro repeat count>
  <#local y = "test">
  <#list 1..count as x>
    ${y} ${count}/${x}: <#nested>
  </#list>
</#macro>
<@repeat count=3>${y!"?"} ${x!"?"} ${count!"?"}</@repeat>
```

will print this:

```
    test 3/1: ? ? ?
```

```
    test 3/2: ? ? ?
    test 3/3: ? ? ?
```

because the `y`, `x` and `count` are the local (private) variables of the macro, and are not visible from outside the macro definition. Furthermore, a different set of local variables is used for each macro call, so this will not cause confusion:

```
<#macro test foo>${foo} (<#nested>) ${foo}</#macro>
<@test foo="A"><@test foo="B"><@test foo="C"/></@test></@test>
```

and will print this:

```
A (B (C () C) B) A
```

Macros with loop variables

Predefined directives like `list` can use so-called loop variables; you should read Defining variables in the template to understand loop variables.

User-defined directives can also have loop variables. For example, let's extend the `do_thrice` directive of the earlier examples so it exposes the current repetition number as a loop variable. As with the predefined directives (as `list`) the *name* of loop variables is given when you call the directive (as `foo` in `<#list foos as foo>...</#list>`), while the *value* of the variables is set by the directive itself.

```
<#macro do_thrice>
  <#nested 1>
  <#nested 2>
  <#nested 3>
</#macro>
<@do_thrice ; x> <#-- user-defined directive uses ";" instead of "as" -->
  ${x} Anything.
</@do_thrice>
```

This will print:

```
  1 Anything.
  2 Anything.
  3 Anything.
```

The syntactical rule is that you pass the actual value of the loop variable for a certain "loop" (i.e. repetition of the nested content) as the parameter of `nested` directive (of course the parameter can by arbitrary expression). The name of the loop variable is specified in the user-defined directive open tag (`<@...>`) after the parameters and a semicolon.

A macro can use more the one loop variable (the order of variables is significant):

```
<#macro repeat count>
  <#list 1..count as x>
    <#nested x, x/2, x==count>
  </#list>
</#macro>
<@repeat count=4 ; c, halfc, last>
  ${c}. ${halfc}<#if last> Last!</#if>
</@repeat>
```

The output will be:

```
  1. 0.5
  2. 1
  3. 1.5
  4. 2 Last!
```

It is not a problem if you specify different number of loop variables in the user-defined directive start-tag (that is, after the semicolon) than with the `nested` directive. If you specify less loop variables after the semicolon, then simply you will not see the last few values that the`nested` directive provides, since there is no loop variable to hold those values. So these are all OK:

```
<@repeat count=4 ; c, halfc, last>
  ${c}. ${halfc}<#if last> Last!</#if>
</@repeat>
<@repeat count=4 ; c, halfc>
  ${c}. ${halfc}
</@repeat>
<@repeat count=4>
  Just repeat it...
</@repeat>
```

If you specify more variables after the semicolon than with the `nested` directive, then the last few loop variables will not be created (i.e. will be undefined in the nested content).

More about user-defined directives and macros

Now you may read the relevant parts of the FreeMarker Reference:

- user-defined directive call
- `macro` directive

You can define methods in FTL as well, see the `function` directive.

Also, you may interested in namespaces: Namespaces. Namespaces help you to organize and reuse your commonly used macros.

Java programmers might want to know that directives (macros are directives) and methods (function-like things) can also be written in Java language, by implementing the `TemplateDirectiveModel` or `TemplateMethodModelEx` interfaces, respectively. Then you can pull in the Java implementations into the template like `<#assign foo = "com.example.FooDirective"?new()>` or `<#assign foo = "com.example.FooMethod"?new()>` on the same place where you would have `<#macro foo ...>` or `<#function foo ...>` otherwise.

Defining variables in the template

Most of the variables that a typical template works with comes from the data-model. But templates can also define variables themselves, usually to hold loops variables, temporary results, macros, etc. Such variables are outside the data-model; modifying the data-model from templates is by design unsupported. Note that each template processing job has its own private set of these variables, which will be thrown away when the template processing job is finished.

You access variables defined in the template the same way as you access variables defined in the data-model root. For example, if you create a variable called "foo" in the template, you can print its value with `${foo}`. If, coincidently, there's a variable called "foo" in the data-model too, the variable created in the template will hide (not overwrite!) it.

There are these kinds of variables that are defined in a template:

- **"plain" variables**: They are accessible from everywhere in the template, or from another templates that was inserted with the `include`directive. You can create and replace these variables with the `assign` directive, or, because macros and functions are just variables, with the `macro` directives and `function` directives.
- **Local variables**: They can only be set inside a macro definition body or function definition body, and are only visible from there, not from other macros or functions called from there. A local variable only exists for the duration of a macro or function call. You can create and replace local variables inside the definition body with the `local` directive. Macro and function parameters are also local variables.
- **Loop variables**: Loop variables are created automatically by directives like `list` (like `x` in `<#list xs as x>...</#list>`), and they only exist between the start-tag and end-tag of the directive. They are only visible directly between these tags, not from macros or functions called from there. As such, they are quite similar to local variables, but they can't be assigned to directly.
- **Global variables**: These should be seldom used. Global variables are shared by all templates, even if they belong to different name spaces because of `import`-ing. Thus, their visibility is similar to data-model variables. They are set via the `global` directive. Global variables hide (but don't overwrite) the data-model variables of the same name.

Example: Create and replace variables with `assign`:

```
<#assign x = 1>  <#-- create variable x -->
${x}
<#assign x = 2> <#-- replace variable x -->
${x}
<#assign x++> <#-- replace variable x -->
${x}
1
2
3
```

In the next example we demonstrate that local variables hide (not overwrite) "plain" variables of the same name, and that loop variables hide (not overwrite) local and "plain" variables of the same name:

```
<#assign x = "plain">
1. ${x}  <#-- we see the plain var. here -->
<@test/>
6. ${x}  <#-- the value of plain var. was not changed -->
<#list ["loop"] as x>
```

```
    7. ${x}  <#-- now the loop var. hides the plain var. -->
    <#assign x = "plain2"> <#-- replaces the plain var, not the loop var. -
->
    8. ${x}  <#-- it still hides the plain var. -->
</#list>
9. ${x}  <#-- now the new value of plain var. becomse visible -->

<#macro test>
  2. ${x}  <#-- we still see the plain var. here -->
  <#local x = "local">
  3. ${x}  <#-- now the local var. hides it -->
  <#list ["loop"] as x>
    4. ${x}  <#-- now the loop var. hides the local var. -->
  </#list>
  5. ${x}  <#-- now we see the local var. again -->
</#macro>
1. plain
  2. plain
  3. local
    4. loop
  5. local
6. plain
    7. loop
    8. loop
9. plain2
```

In the next example we demonstrate that an inner loop variable can hide (not overwrite) an outer loop variable of the same name:

```
<#list ["loop 1"] as x>
  ${x}
  <#list ["loop 2"] as x>
    ${x}
    <#list ["loop 3"] as x>
      ${x}
    </#list>
    ${x}
  </#list>
  ${x}
</#list>
  loop 1
    loop 2
      loop 3
    loop 2
  loop 1
```

When a variable hides the variable from the data-model, you can still read that variable from the data-model using special variable `globals`. For example, assume we have a variable called `user` in the data-model with value "Big Joe":

```
${user}          <#-- prints: Big Joe -->
<#assign user = "Joe Hider">
${user}          <#-- prints: Joe Hider -->
${.globals.user} <#-- prints: Big Joe -->
```

You could also write `.data_model.user` instead, and then not even a `<#global user = "...">` can hide the value in the data-model. However, global variables are often purposely set to override the value coming from the data-model, so using `globals` is a better practice usually.

Auto-escaping and output formats

Page Contents

- Output formats
- Overriding the output format in templates
- Disabling auto escaping
- "Markup output" values
- Further details and tricky cases
  - Non-markup output formats
  - Inserting markup output values from other markups
  - Markup output values and the "+" operator
  - ${...} inside string literals
  - Combined output formats

This is a *detailed* tutorial to auto-escaping and related concepts; for the bare minimum, read this instead.

Note:

The kind of automatic escaping described here requires at least FreeMarker 2.3.24. If you have to use an earlier version, use the deprecated `escape` directive instead.

Output formats

Each template has an associated output format *(a `freemarker.core.OutputFormat` instance)*. The output format dictates the escaping rules, which is applied on all `${...}`-s (and `#{...}`-s) that aren't inside a string literal. It also specifies a MIME type (e.g. `"text/HTML"`) and a canonical name (e.g. `"HTML"`) that the embedding application/framework can use for its own purposes.

It's the programmer's responsibility to associate output format to templates. Furthermore it's recommended that FreeMarker is configured so that templates with `ftlh` and `ftlx` file extensions are automatically associated with the HTML and XML output formats, respectively.

The predefined output formats are:

Name	Description	MIME Type	Default implementation (`freemarker.core.*`)
`HTML`	Escapes `<`, `>`, `&`, `"`, `'` as `<`, `>`, `&`, `"`, `'`	`text/html`	`HTMLOutputFormat.INSTANCE`
`XHTML`	Escapes `<`, `>`, `&`, `"`, `'` as `<`, `>`, `&`, `"`, `'`	`application/xhtml+xml`	`XHTMLOutputFormat.INSTANCE`
`XML`	Escapes `<`, `>`, `&`, `"`, `'` as `<`, `>`, `&`, `"`, `'`	`application/xml`	`XMLOutputFormat.INSTANCE`
`RTF`	Escapes `{`, `}`, `\` as `\{`, `\}`, `\`	`application/`	`RTFOutputFormat.INSTANCE`

Name	Description	MIME Type	Default implementation (`freemarker.core.*`)
	`\`	`rtf`	
`undefined`	Doesn't escape. Prints markup output values (concept explained later) from other output formats as is. The default output format used when no output format was explicitly set in the configuration.	None (`null`)	`UndefinedOutputFormat.INSTANCE`
`plainText`	Doesn't escape.	`text/plain`	`PlainTextOutputFormat.INSTANCE`
`JavaScript`	Doesn't escape.	`application/javascript`	`JavaScriptOutputFormat.INSTANCE`
`JSON`	Doesn't escape.	`application/json`	`JSONOutputFormat.INSTANCE`
`CSS`	Doesn't escape.	`text/css`	`CSSOutputFormat.INSTANCE`

The programmers can add their your own output formats, so this is maybe not all the output formats in your application!

Overriding the output format in templates

Especially in legacy applications, you will often find that the output format is `undefined` (you can check that with `${.output_format}`), and so no automatic escaping is happening. In other cases, a common output format (like HTML) is set for all templates, but a few templates need a different output format. In any case, the output format of a template can be enforced in the the `ftl` header:

```
<#ftl output_format="XML">
${"'"}  <#-- Prints: &apos; -->
```

Above, the output format was referred by its name shown in the earlier table *(looked up via `Configuration.getOutputFormat(String name)`, actually)*.

Note:

If escaping doesn't happen after adding the above `ftl` header, then `<#ftl output_format="XML" auto_esc=true>` might helps (and that means that FreeMarker was configured to use "disable" auto-escaping *policy*, which is generally not recommended).

The output format can also be applied to only a section of a template, like:

```
<#-- Let's assume we have "HTML" output format by default. -->
${"'"}  <#-- Prints: &#39; -->
<#outputformat "XML">
  ${"'"}  <#-- Prints: &apos; -->
</#outputformat>
${"'"}  <#-- Prints: &#39; -->
```

Basically, each position in a template has an associated output format, and as you saw above, it might not be the same everywhere in the template. This association sticks to the positions and won't change as the template executes. So if, for example, you call a macro from inside an `outputformat` block and the called macro is defined outside that block, it won't get the output format of it. Or, if you have a macro that's defined in a template with HTML output format, no mater from where you call it, that macro will always execute with HTML output format. This is like if you were coloring each characters of the template files by output format in the text editor, and then later when the templates are executed, it only considers the color of the statement being executed. This gives you firm control over the output format and hence escaping; you don't have to consider the possible execution paths that can lead to a point.

Disabling auto escaping

For a single interpolation you can disable auto-escaping with `?no_esc`:

```
<#-- Let's assume we have "HTML" output format by default. -->
${'<b>test</b>'}  <#-- prints: &lt;b&gt;test&lt;/b&gt; -->
${'<b>test</b>'?no_esc}  <#-- prints: <b>test</b> -->
```

You can also disable auto escaping for a whole section with the `noautoesc` directive:

```
${'&'}  <#-- prints: &amp; -->
<#noautoesc>
  ${'&'}  <#-- prints: & -->
  ...
  ${'&'}  <#-- prints: & -->
</#noautoesc>
${'&'}  <#-- prints: &amp; -->
```

Just like `outputformat`, this only applies to the part that's literally inside the block ("coloring" logic).

Auto-escaping can also be disabled for the whole template in the `ftl` header. It can then be re-enabled for a section with the `autoesc`directive:

```
<#ftl autoesc=false>
${'&'}  <#-- prints: & -->
<#autoesc>
  ${'&'}  <#-- prints: &amp; -->
  ...
  ${'&'}  <#-- prints: &amp; -->
</#autoesc>
${'&'}  <#-- prints: & -->
```

You can also force escaping for an individual interpolation when escaping is disabled, with `?esc`:

```
<#ftl autoesc=false>
${'&'}  <#-- prints: & -->
${'&'?esc}  <#-- prints: &amp; -->
```

Naturally, both `autoesc` and `?esc` works inside `noautoesc` blocks too.

"Markup output" values

In FTL, values have type, like string, number, boolean, etc. One such type is called "markup output". A value of that type is a piece of text that's already in the output format (like HTML), and hence needs no further escaping. We have already produced such values earlier:

- *s*`?esc` creates a markup output value out of a string value by escaping all special characters in it.
- *s*`?no_esc` creates a markup output value out of a string value by assuming that the string already stores markup and so needs no further escaping

These can be useful outside `${...}` too. For example, here the caller of the `infoBox` macro can decide if the message is plain text (hence needs escaping) or HTML (hence it mustn't be escaped):

```
<#-- We assume that we have "HTML" output format by default. -->

<@infoBox "Foo & bar" />
<@infoBox "Foo <b>bar</b>"?no_esc />

<#macro infoBox message>
  <div class="infoBox">
    ${message}
  </div>
</#macro>
  <div class="infoBox">
    Foo &amp; bar
  </div>
  <div class="infoBox">
    Foo <b>bar</b>
  </div>
```

Another case where you get a markup output value is output capturing:

```
<#-- We assume that we have "HTML" output format by default. -->
<#assign captured><b>Test</b></#assign>
Just a string: ${"<b>Test</b>"}
Captured output: ${captured}
Just a string: &lt;b&gt;Test&lt;/b&gt;
Captured output: <b>Test</b>
```

Because the captured output is markup output, it wasn't auto-escaped.

It's important that markup output values aren't strings, and aren't automatically coerced to strings. Thus `?upper_case`, `?starts_with` etc., will give an error with them. You won't be able to pass them to Java methods for `String` parameters either. But sometimes you need the markup that's behind the value as a string, which you can get as *markupOutput*`?markup_string`. Be sure you know what you are doing though. Applying

string operations on markup (as opposed to on plain text) can result in invalid markup. Also there's the danger of unintended double escaping.

```
<#-- We assume that we have "HTML" output format by default. -->

<#assign markupOutput1="<b>Test</b>"?no_esc>
<#assign markupOutput2="Foo & bar"?esc>

As expected:
${markupOutput1}
${markupOutput2}

Possibly unintended double escaping:
${markupOutput1?markup_string}
${markupOutput2?markup_string}
As expected:
<b>Test</b>
Foo &amp; bar

Possibly unintended double escaping:
&lt;b&gt;Test&lt;/b&gt;
Foo &amp;amp; bar
```

Further details and tricky cases

Non-markup output formats

An output format is said to be a non-markup format if it defines no escaping rules. Examples of such output formats are the `undefined`format and the `plainText` format.

These formats produce no markup output values, hence you can't use `?esc` or `?no_esc` when they are the current format. You can use output capturing (like `<#assign captured>...</#assign>`), but the resulting value will be a string, not a markup output value.

Furthermore, you aren't allowed to use the `autoesc` directive or `<#ftl auto_esc=true>` when the current output format is non-markup.

Using constructs that aren't supported by the current output format will give parse-time error.

Inserting markup output values from other markups

Each markup output value has an associated output format. When a markup output value is inserted with `${...}` (or `#{...}`), it has to be converted to the current output format at the point of insertion (if they differ). As of this writing (2.3.24), such output format conversion will only be successful if the value to convert was created by escaping plain text:

```
<#-- We assume that we have "HTML" output format by default. -->

<#assign mo1 = "Foo's bar {}"?esc>
HTLM: ${mo1}
XML:  <#outputformat 'XML'>${mo1}</#outputformat>
RTF:  <#outputformat 'RTF'>${mo1}</#outputformat>

<#assign mo2><p>Test</#assign>
HTML: ${mo2}
```

```
XML:  <#attempt><#outputformat
'XML'>${mo2}</#outputformat><#recover>Failed</#attempt>
RTF:  <#attempt><#outputformat
'RTF'>${mo2}</#outputformat><#recover>Failed</#attempt>
HTLM: Foo&#39;s bar {}
XML:  Foo&apos;s bar {}
RTF:  Foo's bar \{\}

HTML: <p>Test
XML:  Failed
RTF:  Failed
```

But, an output format can also chose to insert pieces of other output formats as is, without converting them. Among the standard output formats, `undefined` is like that, which is the output format used for templates for which no output format was specified in the configuration:

```
<#-- We assume that we have "undefined" output format here. -->

<#outputformat "HTML"><#assign htmlMO><p>Test</#assign></#outputformat>
<#outputformat "XML"><#assign xmlMO><p>Test</p></#assign></#outputformat>
<#outputformat "RTF"><#assign rtfMO>\par Test</#assign></#outputformat>
HTML: ${htmlMO}
XML:  ${xmlMO}
RTF:  ${rtfMO}
HTML: <p>Test
XML:  <p>Test</p>
RTF:  \par Test
```

Markup output values and the "+" operator

As you certainly know, if one of the sides of the `+` operator is a string then it does concatenation. If there's a markup output value in one side, the other side gets promoted to markup output value of the same output format (if it's not already that), by escaping its string value, and finally the two markups are concatenated to form a new markup output value. Example:

```
<#-- We assume that we have "HTML" output format by default. -->
${"<h1>"?no_esc + "Foo & bar" + "</h1>"?no_esc}
<h1>Foo &amp;amp; bar</h1>
```

If the two sides of the `+` operator are markup values of different output formats, the right side operand is converted to the output format of the left side. If that's not possible, then the left side operand is converted to the output format of the right side. If that isn't possible either, that's an error. (See the limitations of conversions here.)

${...} inside string literals

When `${...}` is used inside string *expressions* (e.g., in `<#assign s = "Hello ${name}!">`), it's just a shorthand of using the `+` operator (`<#assign s = "Hello" + name + "!">`). Thus, `${...}`-s inside string expressions aren't auto-escaped, but of course when the resulting concatenated string is printed later, it will be possibly auto-escaped.

```
<#-- We assume that we have "HTML" output format by default. -->
<#assign name = "Foo & Bar">

<#assign s = "<p>Hello ${name}!">
${s}
<p>Hello ${name}!
```

```
To prove that s didn't contain the value in escaped form:
${s?replace('&'), 'and'}
&lt;p&gt;Hello Foo &amp;amp; Bar!
<p>Hello Foo &amp;amp; Bar!

To prove that "s" didn't contain the value in escaped form:
&lt;p&gt;Hello Foo and Bar!
```

Combined output formats

Combined output formats are output formats that are created ad-hoc from other output formats by nesting them into each other, so that the escaping of both output formats are applied. They are discussed here...

White-space handling

Page Contents

- White-space stripping
- Using compress directive

The control of the white-space in a template is a problem that to some extent haunts every template engine in the business.

Let see this template. I have marked the components of template with colors: text, interpolation, FTL tag. With the *[BR]*-s I visualize the line breaks.

```
<p>List of users:[BR]
<#assign users = [{"name":"Joe",         "hidden":false},[BR]
                  {"name":"James Bond", "hidden":true},[BR]
                  {"name":"Julia",      "hidden":false}]>[BR]
<ul>[BR]
<#list users as user>[BR]
  <#if !user.hidden>[BR]
  <li>${user.name}[BR]
  </#if>[BR]
</#list>[BR]
</ul>[BR]
<p>That's all.
```

If FreeMarker were to output all text as is, the output would be:

```
<p>List of users:[BR]
[BR]
<ul>[BR]
[BR]
  [BR]
  <li>Joe[BR]
  [BR]
[BR]
  [BR]
[BR]
  [BR]
  <li>Julia[BR]
  [BR]
[BR]
</ul>[BR]
<p>That's all.
```

You have a lot of unwanted spaces and line breaks here. Fortunately neither HTML nor XML is typically white-space sensitive, but this amount of superfluous white-space can be annoying, and needlessly increases the size of produced HTML. Of course, it is even bigger problem when outputting white-space-sensitive formats.

FreeMarker provides the following tools to cope with this problem:

- Tools to ignore certain white-space of the template files *(parse time white-space removal)*:
  - White-space stripping: This feature automatically ignores typical superfluous white-space around FTL tags. It can be enabled or disabled on per template manner.

    - Trimmer directives: `t`, `rt`, `lt`. With these directives you can explicitly tell FreeMarker to ignore certain white-space. Read the reference for more information.
    - `ftl` parameter `strip_text`: This removes all top-level text from the template. It is useful for templates that contain macro definitions only (and some other non-outputting directives), because it removes the line-breaks that you use between the macro definitions and between the other top-level directives to improve the readability of the template.
- Tools that remove white-space from the output *(on-the-fly white-space removal)*:
    - `compress` directive.

White-space stripping

If this feature is enabled for a template, then it automatically ignores (i.e. does not print to the output) two kind of typical superfluous white-space:

- Indentation white-space, and trailing white-space at the end of the line (includes the line break) will be ignored in lines that contains only FTL tags (e.g. `<@myMacro/>`, `<#if ...>`) and/or FTL comments (e.g. `<#-- blah -->`), apart from the the ignored white-space itself. For example, if a line contains only an `<#if ...>`, then the indentation before the tag and the line break after the tag will be ignored. However, if the line contains `<#if ...>x`, then the white-space in that line will not be ignored, because of the `x`, as that is not FTL tag. Note that according these rules, a line that contains `<#if ...><#list ...>` is subject to white-space ignoring, while a line that contains `<#if ...> <#list ...>` is not, because the white-space between the two FTL tags is embedded white-space, not indentation or trailing white-space.
- White-space sandwiched between the following directives is ignored: `macro`, `function`, `assign`, `global`, `local`, `ftl`, `import`, but only if there is *only* white-space and/or FTL comments between the directives. In practice it means that you can put empty lines between macro definitions and assignments as spacing for better readability, without printing needless empty lines (line breaks) to the output.

The output of the last example with white-space stripping enabled will be:

```
<p>List of users:[BR]
<ul>[BR]
  <li>Joe[BR]
  <li>Julia[BR]
</ul>[BR]
<p>That's all.
```

This is because after stripping the template becomes the following; the ignored text is not colored:

```
<p>List of users:[BR]
<#assign users = [{"name":"Joe",         "hidden":false},[BR]
                  {"name":"James Bond", "hidden":true},[BR]
                  {"name":"Julia",      "hidden":false}]>[BR]
<ul>[BR]
<#list users as user>[BR]
  <#if !user.hidden>[BR]
  <li>${user.name}[BR]
  </#if>[BR]
</#list>[BR]
</ul>[BR]
```

```
<p>That's all.
```

White-space stripping can be enabled/disabled in per template manner with the `ftl` directive. If you don't specify this with the `ftl` directive, then white-space stripping will be enabled or disabled depending on how the programmer has configured FreeMarker. The factory default is white-space stripping enabled, and the programmers probably left it so (*recommended*). *Note that enabling white-space stripping does notdegrade the performance of template execution; white-space stripping is done during template loading.*

White-space stripping can be disabled for a single line with the `nt` directive (for No Trim).

Using compress directive

Another solution is to use the `compress` directive. As opposed to white-space stripping, this works directly on the generated output, not on the template. That is, it will investigate the printed output on the fly, and does not investigate the FTL program that creates the output. It aggressively removes indentations, empty lines and repeated spaces/tabs (for more information read the reference). So the output of:

```
<#compress>
<#assign users = [{"name":"Joe",         "hidden":false},
                  {"name":"James Bond", "hidden":true},
                  {"name":"Julia",      "hidden":false}]>
List of users:
<#list users as user>
  <#if !user.hidden>
  - ${user.name}
  </#if>
</#list>
That's all.
</#compress>
```

will be:

```
List of users:
- Joe
- Julia
That's all.
```

Note that `compress` is totally independent of white-space stripping. So it is possible that the white-space of template is stripped, and later the produced output is `compress`-ed.

Also, by default a user-defined directve called `compress` is available in the data-model (due to backward compatibility). This is the same as the directive, except that you may optionally set the `single_line` parameter, which will remove all intervening line breaks. If you replace`<#compress>...</#compress>` on the last example with `<@compress single_line=true>...</@compress>`, then you get this output:

```
List of users: - Joe - Julia That's all.
```

Alternative (square bracket) syntax

Note:

This feature exists since FreeMarker 2.3.4.

FreeMarker supports an alternative syntax, where `[` and `]` is used instead of `<` and `>` in FreeMarker directives and comments, for example:

- Calling predefined directive: `[#list animals as animal]...[/#list]`
- Calling user-defined directive: `[@myMacro /]`
- Comment: `[#-- the comment --]`

To use the alternative syntax instead of the default one, start the template with the `ftl` directive using the alternative syntax. If you don't know what is the `ftl` directive, just start the template with `[#ftl]`, and remember that it should be the very first thing in the file (except thatwhite-space can precede it). For example, this is how the last example of the Getting Started looks with the alternative syntax (assuming it's a complete template, not just a fragment):

```
[#ftl]
<p>We have these animals:
<table border=1>
  <tr><th>Name<th>Price
  [#list animals as animal]
  <tr>
    <td>
      [#if animal.size == "large"]<b>[/#if]
      ${animal.name}
      [#if animal.size == "large"]</b>[/#if]
    <td>${animal.price} Euros
  [/#list]
</table>
```

The alternative (square bracket) and the default (angle bracket) syntax are mutually exclusive within a template. That is, either the whole template uses alternative syntax, or the whole template uses the default syntax. If the template uses alternative syntax, things like `<#if...>` are count as static text, not as FTL tags. Similarly, if the template uses the default syntax, things like `[#if ...]` count as static text, not as FTL tags.

If you start the file with `[#ftl ...]` (where the ... stands for the optional parameters; of course `[#ftl]` works too) the file will surely use the alternative (square bracket) syntax. If you start the file with `<#ftl ...>` the file will surely use the normal (angle bracket) syntax. If there is no `ftl` directive in the file, then the programmer decides what the syntax will be by configuring FreeMarker *(programmers see `Configuration.setTagSyntax(int)` in the API javadocs)*. Most probably the programmers use the factory default however. The factory default in 2.3.x is using the normal syntax. The factory default in 2.4.x will be auto-detection, which means that the first FreeMarker tag determines the syntax (it can be anything, not just `ftl`).

attempt, recover

Page Contents

- Synopsis
- Description

Synopsis

```
<#attempt>
  attempt block
<#recover>
  recover block
</#attempt>
```

Where:

- *attempt block*: Template block with any content. This will be always executed, but if an error occurs during that, all output from this block is rolled back, and the *recover block* will be executed.
- *recover block*: Template block with any content. This will be executed only if there was an error during the execution of the*attempt block*. You may print an error messages here and such.

The *recover* is mandatory. `attempt`/`recover` can be nested freely into other *attempt block*s or *recover block*s.

Note:

The format shown here is supported starting from 2.3.3; earlier it was `<#attempt>...<#recover>...</#recover>`, which is still supported for backward compatibility. Furthermore, these directives were introduced with FreeMarker 2.3.1, so they aren't exist in 2.3.

Description

These directives are used if you want the page successfully outputted even if the outputting of a certain part of the page fails. If an error occurs during the execution of the *attempt block*, then the template execution is not aborted, but the *recover block* is executed instead of the *attempt block*. If no error occurs during the execution of the *attempt block*, then the *recover block* is ignored. A simple example:

```
Primary content
<#attempt>
  Optional content: ${thisMayFails}
<#recover>
  Ops! The optional content is not available.
</#attempt>
Primary content continued
```

If the `thisMayFails` variable doesn't exist, then the output is:

```
Primary content
  Ops! The optional content is not available.
Primary content continued
```

If the `thisMayFails` variable exists and it's value is `123`, then the output is:

```
Primary content
  Optional content: 123
Primary content continued
```

The *attempt block* has an all-or-none semantic: either the entire content of the *attempt block* is output (when there was no error), or no output at all results from the execution of the *attempt block* (when there was an error). For example, above, the failure happens after ``Optional content: '' was printed, still it is not there in the output before the ``Ops!''. (*This is implemented with the aggressive buffering of the output inside the attempt block. Not even the `flush` directive will send the output to the client.*)

To prevent misunderstandings coming from the above example: `attempt`/`recover` is not (only) for handling undefined variables (for that use missing value handler operators). It can handle all kind of errors that occurs when the block is executed (i.e. not syntactical errors, which are detected earlier). And it's meant to enclose bigger template fragments, where error can occur at various points. For example, you have a part in your template that deals with printing advertisements, but that's not the primary content of the page, so you don't want your whole page be down just because some error occurs with the printing of the advertisements (say, because of a temporal database server faliure). So you put the whole advertisement printing into an *attempt block*.

In some environments programmers configure FreeMarker so that it doesn't abort template execution for certain errors, but continues execution, possibly after printing some error indicator to the output (*see more here...*). The `attempt` directive doesn't consider such suppressed errors as errors.

Inside a *recover block* the error message of the error is available with the `error` special variable. Don't forget that references to special variable are started with dot (for example: `${.error}`).

Errors occurring during template execution are always logged, even if they occur inside an attempt block.

compress

Page Contents

- Synopsis
- Description

Synopsis

```
<#compress>
  ...
</#compress>
```

Description

The compress directive is useful for removing superfluous white-space when you use a white-space insensitive format (e.g. HTML or XML). It captures the output generated inside its body (i.e. between its start-tag and end-tag), and reduces all unbroken white-space sequences to a single white-space character. The inserted character will be a line break if the replaced sequence contains line breaks, or a space otherwise. The very first and very last unbroken white-space sequences will be completely removed.

```
<#assign x = "    moo  \n\n   ">
(<#compress>
  1 2  3   4    5
  ${moo}
  test only

  I said, test only

</#compress>)
```

will output:

```
(1 2 3 4 5
moo
test only
I said, test only)
```

ftl

Page Contents

- Synopsis
- Description

Synopsis

```
<#ftl param1=value1 param2=value2 ... paramN=valueN>
```

Where:

- *param1*, *param2*, ...etc.: Name of the parameter. Not an expression. Allowed parameters are: `encoding`, `strip_whitespace`,`strip_text`, ...etc. See below.
- *value1*, *value2*, ...etc.: The value of parameter. This must be a constant expression (as `true`, or `"ISO-8859-5"`, or `{x:1, y:2}`). It can't use variables.

Description

Tells information about the template for FreeMarker and for other tools, also helps programs to automatically detect if a text file is an FTL file. This directive, if present, must be the very first thing in the template. Any white-space before this directive will be ignored. The old-syntax (`#`-less) format of this directive is not supported.

The settings (encoding, white-space stripping, etc.) given here has the highest precedence, that is, they will be used for the template regardless of any FreeMarker configuration settings.

Possible parameters:

- `attributes`: This is a hash that associates arbitrary attributes (name-value pairs) to the template. The values of the attributes can be of any type (string, number, sequence... etc.). FreeMarker doesn't try to understand the meaning of the attributes. It's up to the application that encapsulates FreeMarker (as a Web application framework). Thus, the set of allowed attributes and their semantic is application (Web application framework) dependent. *Programmers: you can get the attributes associated with a `Template` object with its `getCustomAttributeNames` and `getCustomAttribute` methods (inherited from `freemarker.core.Configurable`). As the template attributes are associated with the `Template` object when the template is parsed, the attributes can be read anytime, the template need not be executed. The methods mentioned return the attribute values unwrapped, that is, with FreeMarker independent type as `java.util.List`.*
- `auto_esc`: A boolean constant to turn auto-escaping on or off. It depends on the `auto_escaping_policy` of the FreeMarker configuration, but usually auto-escaping will be by default on, if the current output format uses auto-escaping by default. So you mostly use this to disable auto-escaping (`false` value). An attempt to use `true` value when the current output format is a non-markup output format (which hence can't escape) will cause parse-time error. Note that you can turn auto-escaping on/off for only a part of the template with the `autoesc` and `noautoesc` directives.
- `encoding`: With this you can specify the encoding (charset) of the template in the template file itself. *(That is, this will be the `encoding` setting of the newly created `Template`, and not even the `encoding` parameter to `Configuration.getTemplate` can override it)*. Note however, that FreeMarker will try to find and interpret the `ftl` directive first with the automatically guessed encoding

(which depends on the FreeMarker configuration set by the programmers), and only then realizes if the `ftl` directive dictates something different, and re-read the template with the new encoding. Thus, the template must be valid FTL until the end of `ftl` tag with the encoding tried first. The valid values of this parameter are MIME-preferred charset names from the IANA Charset Registry, like ISO-8859-5, UTF-8 or Shift_JIS.

- `ns_prefixes`: This is a hash that associates prefixes with node namespaces. For example: `{"e":"http://example.com/ebook", "vg":"http://example.com/vektorGraphics"}`. This is mostly used with XML processing where the prefixes can be used in XML queries, but it also influences the working of directives `visit` and `recurse`. Only one prefix can be registered for the same node namespace (otherwise an error will occur), so there is one-to-one relation between prefixes and node namespaces. Prefixes `D` and `N`are reserved. If you register prefix `D`, then other than you associate the node namespace with prefix `D`, you also set the default node namespace. Prefix `N` can't be registered; it is used to denote nodes with no node namespace in certain places, when (and only when) prefix `D` is registered. (To see the usage of default node namespace, `N`, and prefixes in general, see the part about XML processingand `visit` and `recurse` in the reference.) The effect of `ns_prefixes` is limited to a single FTL namespace, namely, to the FTL namespace that was created for the template. This also means that `ns_prefixes` has effect only when an FTL namespace is created for the template that contains it, otherwise the `ns_prefixes` parameter has no effect. An FTL namespace is made for a template when: (a) the template is the ``main'' template, that is, it is not invoked as a result of an `<#include ...>`, but it is directly invoked (*with the `process` Java method of class `Template` or `Environment`*); (b) the template is invoked directly with `<#import ...>`.
- `output_format`: Specifies the output format of this template. This must be a string literal, which refers to the name of the output format. See the table of predefined output formats here. Other names can exist if the programmers has added them via the`registered_custom_output_formats` configuration setting (`Configuration.setRegisteredCustomOutputFormats(...)`). The referred output format must be known by the `Configuration`, or else a parse-time error will occur. The name can also refer to a so called combined output format as `"`*outerFormatName*`{`*innerFormatName*`}"`; see more about combined output formats here.
- `strict_syntax`: This turns on/off ``strict syntax''. Valid values are the boolean constants `true` and `false`. (And for backward compatibility, strings `"yes"`, `"no"`, `"true"`, `"false"`). The default value (i.e. when you don't use this parameter) depends on the FreeMarker configuration set by the programmers, but it should be `true` for new projects. *(Programmers: you must set explicitly this setting to `true` for this: `config.setStrictSyntaxMode(true);`)* For more information read: Deprecated FTL constructs/Old FTL syntax
- `strip_text`: When enabled, all top-level text in a template is removed when the template is parsed. This does not affect text within macros, directives, or interpolations. Valid values are the boolean constants `true` and `false`. (And for backward compatibility, strings`"yes"`, `"no"`, `"true"`, `"false"`). The default value (i.e. when you don't use this parameter) is `false`.
- `strip_whitespace`: This enables/disables white-space stripping. Valid values are the boolean constants `true` and `false`. (And for backward compatibility, strings `"yes"`, `"no"`, `"true"`, `"false"`). The default value (i.e. when you don't use this parameter) depends on the FreeMarker configuration set by the programmers, but it should be `true` for new projects.

Note:

As of FreeMarker 2.3.23, you can use camel case instead of snake case for parameter names, like `outputFormat` instead of `output_format`. But know that then within the same template, FreeMarker will enforce the usage of camel case for all identifiers that are part of the template language (user defined names are not affected).

This directive also determines if the template uses angle bracket syntax (e.g. `<#include 'foo.ftl'>`) or square bracket syntax (e.g. `[#include 'foo.ftl']`). Simply, the syntax used for this directive will be the syntax used for the whole template, regardless of the FreeMarker configuration settings.

function, return

Page Contents

- Synopsis
- Description

Synopsis

```
<#function name param1 param2 ... paramN>
  ...
  <#return returnValue>
  ...
</#function>
```

Where:

- *name*: name of method variable (not expression)
- *param1*, *param2*, ...etc.: the name of the local variables store the parameter values (not expression), optionally followed by = and the default value (that's an expression).
- *paramN*, the last parameter, may optionally include a trailing ellipsis (...), which indicates the macro takes a variable number of parameters. Local variable *paramN* will be a sequence of the extra parameters.
- *returnValue*: the expression that calculates the value of the method call.

The `return` directive can be used anywhere and for any times between the `<#function ...>` and `</#function>`.

Parameters without default value must precede parameters with default value (*paramName=defaultValue*).

Description

Creates a method variable (in the current namespace, if you know namespace feature). This directive works in the same way as the `macro`directive, except that `return` directive *must* have a parameter that specifies the return value of the method, and that attempts to write to the output will be ignored. If the `</#function>` is reached (i.e. there was no `return` *returnValue*), then the return value of the method is an undefined variable.

Example 1: Creating a method that calculates the average of two numbers:

```
<#function avg x y>
  <#return (x + y) / 2>
</#function>
${avg(10, 20)}
```

will print:

```
15
```

Example 2: Creating a method that calculates the average of multiple numbers:

```
<#function avg nums...>
  <#local sum = 0>
  <#list nums as num>
    <#local sum += num>
  </#list>
  <#if nums?size != 0>
    <#return sum / nums?size>
  </#if>
</#function>
${avg(10, 20)}
${avg(10, 20, 30, 40)}
${avg()!"N/A"}
```

will print:

```
15
25
N/A
```

if, else, elseif

Page Contents

- Synopsis
- Description

Synopsis

```
<#if condition>
  ...
<#elseif condition2>
  ...
<#elseif condition3>
  ...
...
<#else>
  ...
</#if>
```

Where:

- *condition*, *condition2*, ...etc.: Expression evaluates to a boolean value.

The `elseif`-s and the `else` are optional.

Camel case name variant: `elseIf`

Description

You can use `if`, `elseif` and `else` directives to conditionally skip a section of the template. The *condition*-s must evaluate to a boolean value, or else an error will abort template processing. The `elseif`-s and `else`-s must occur inside `if` (that is, between the `if` start-tag and end-tag). The `if` can contain any number of `elseif`-s (including 0) and at the end optionally one `else`. Examples:

`if` with 0 `elseif` and no `else`:

```
<#if x == 1>
  x is 1
</#if>
```

`if` with 0 `elseif` and `else`:

```
<#if x == 1>
  x is 1
<#else>
  x is not 1
</#if>
```

`if` with 2 `elseif` and no `else`:

```
<#if x == 1>
  x is 1
<#elseif x == 2>
  x is 2
<#elseif x == 3>
  x is 3
</#if>
```

`if` with 3 `elseif` and `else`:

```
<#if x == 1>
  x is 1
<#elseif x == 2>
  x is 2
<#elseif x == 3>
  x is 3
<#elseif x == 4>
  x is 4
<#else>
  x is not 1 nor 2 nor 3 nor 4
</#if>
```

To see more about boolean expressions, see: Template Author's Guide/The Template/Expressions.

You can nest `if` directives (of course):

```
<#if x == 1>
  x is 1
  <#if y == 1>
    and y is 1 too
  <#else>
    but y is not
  </#if>
<#else>
  x is not 1
  <#if y < 0>
    and y is less than 0
  </#if>
</#if>
```

Note:

When you want to test if `x > 0` or `x >= 0`, writing `<#if x > 0>` and `<#if x >= 0>` is WRONG, as the first `>` will close the `#if` tag. To work that around, write `<#if x gt 0>` or `<#if gte 0>`. Also note that if the comparison occurs inside parentheses, you will have no such problem, like `<#if foo.bar(x > 0)>` works as expected.

1. Template Reuse

1.1. Reuse common template fragments

Create a new folder called *templates* inside the folder of the *com.vogella.freemarker.first* package. Inside that, create the following file with name *helloworld.ftl*.

```
<html>
<head>
  <title>${title}</title>
</head>
<body>
  <h1>${title}</h1>

  <p>${exampleObject.name} by ${exampleObject.developer}</p>

  <ul>
    <#list systems as system>
      <li>${system_index + 1}. ${system.name} from
${system.developer}</li>
    </#list>
  </ul>

</body>
</html>
```

Create the following class which demonstrates the usage of Java objects in templates.

```
package com.vogella.freemarker.first;

public class ValueExampleObject {

  private String name;
  private String developer;

  public ValueExampleObject(String name, String developer) {
    this.name = name;
    this.developer = developer;
  }

  public String getName() {
    return name;
  }

  public String getDeveloper() {
    return developer;
  }

}
```

Create the following class which creates the input for this template and creates the output.

```
package com.vogella.freemarker.first;
```

```
import java.io.File;
import java.io.FileWriter;
import java.io.OutputStreamWriter;
import java.io.Writer;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

import freemarker.template.Configuration;
import freemarker.template.Template;
import freemarker.template.TemplateExceptionHandler;
import freemarker.template.Version;

public class MainTest {

  public static void main(String[] args) throws Exception {

    // 1. Configure FreeMarker
    //
    // You should do this ONLY ONCE, when your application starts,
    // then reuse the same Configuration object elsewhere.

    Configuration cfg = new Configuration();

    // Where do we load the templates from:
    cfg.setClassForTemplateLoading(MainTest.class, "templates");

    // Some other recommended settings:
    cfg.setIncompatibleImprovements(new Version(2, 3, 20));
    cfg.setDefaultEncoding("UTF-8");
    cfg.setLocale(Locale.US);

cfg.setTemplateExceptionHandler(TemplateExceptionHandler.RETHROW_HANDLER
);

    // 2. Proccess template(s)
    //
    // You will do this for several times in typical applications.

    // 2.1. Prepare the template input:

    Map<String, Object> input = new HashMap<String, Object>();

    input.put("title", "Vogella example");

    input.put("exampleObject", new ValueExampleObject("Java object",
"me"));

    List<ValueExampleObject> systems = new
ArrayList<ValueExampleObject>();
    systems.add(new ValueExampleObject("Android", "Google"));
    systems.add(new ValueExampleObject("iOS States", "Apple"));
    systems.add(new ValueExampleObject("Ubuntu", "Canonical"));
    systems.add(new ValueExampleObject("Windows7", "Microsoft"));
    input.put("systems", systems);

    // 2.2. Get the template
```

```
    Template template = cfg.getTemplate("helloworld.ftl");

    // 2.3. Generate the output

    // Write output to the console
    Writer consoleWriter = new OutputStreamWriter(System.out);
    template.process(input, consoleWriter);

    // For the sake of example, also write output into a file:
    Writer fileWriter = new FileWriter(new File("output.html"));
    try {
      template.process(input, fileWriter);
    } finally {
      fileWriter.close();
    }

  }
}
```

5. Useful FTL tricks

5.1. Reuse common template fragments

When you find yourself copy-pasting common parts between templates a lot, you should probably use macros.

Continuing our last example, create a new folder called *lib* inside the *templates* directory, and there create a file called *utils.ftl*, with this content:

```
<#macro page>
  <html>
  <head>
    <title>${title}</title>
  </head>
  <body>
    <h1>${title}</h1>

    <#-- This processes the enclosed content:  -->
    <#nested>
  </body>
  </html>
</#macro>

<#macro otherExample p1 p2>
  <p>The parameters were: ${p1}, ${p2}</p>
</#macro>
```

Now you can simplify *helloworld.ftl* like this:

```
<#import "lib/utils.ftl" as u>

<@u.page>
  <p>${exampleObject.name} by ${exampleObject.developer}</p>

  <ul>
    <#list systems as system>
      <li>${system_index + 1}. ${system.name} from
${system.developer}</li>
    </#list>
  </ul>

  <#-- Just another example of using a macro: -->
  <@u.otherExample p1=11 p2=22 />
</@u.page>
```

Another way of reusing template fragments is moving the common fragment into its own ftl file. Then just insert it with `<#include "lib/myfragment.ftl">`. This is less flexible than macros, but simpler in concept: it mimics copy-pasting.

5.2. Variables

You can define and assign content to variables inside the FTL files for easy reuse.

```
<#assign var_link = "http://www.vogella.com/people/larsvogel.html">

<a href="${var_link}">About Lars Vogel</a>
```

5.3. Handling null/undefined values

FreeMarker requires you to provide an explicit default for variables, so avoid values that are `null` or undefined:

```
<!-- Acts like if the color was N/A if there's no color: -->
<p>Color: ${color!'N/A'}</p>

<!-- Avoid the whole color row if there's no color: -->
<#if color??>
  <p>Color: ${color}</p>
</#if>
```

5.4. Escape

When generating HTML, it's important to escape `<`, `&`, etc. in values that were not meant to store HTML and can contain these problematic characters. You can apply such escaping like `${message?html}`. You can also ask FreeMarker to add `?html` to all `${}` -s in a section like this:

```
<#escape x as x?html>
  <p>Sender: ${from}
  <p>Title: ${title}
  <p>Message: ${body}
</#escape>
```

It's important to understand that `#escape` only affects the `${}` bits that are inside the enclosed section in the template file when you look at it in a text editor. That means, `${}` embracements which are in other templates or macros called from there, won't be affected.